NEA - Group Creator for School Projects

Amelie Machin

Contents

1 Analysis **2**
1.1 Problem statement . . . 2
1.2 Research . . . 2
1.2.1 Sources of inspiration . . . 2
1.2.2 Algorithm . . . 3
1.2.3 Inputting data . . . 4
1.2.4 Accounts . . . 4
1.2.5 Internal data storage . . . 5
1.2.6 External data storage . . . 5
1.2.7 GUI library . . . 5
1.3 Dialogue . . . 6
1.3.1 Initial interview . . . 6
1.3.2 Follow-up questions . . . 7
1.4 Objectives . . . 8
1.5 Modelling . . . 13

2 Design **17**
2.1 GUI . . . 17
2.2 Databases . . . 24
2.3 Inputs and outputs . . . 24
2.4 Algorithm . . . 26
2.4.1 Pseudocode . . . 26
2.4.2 Fitness algorithm . . . 28
2.4.3 Diagrams . . . 29

3 Technical solution **33**
3.1 main.py . . . 34
3.2 geneticAlgo.py . . . 62
3.3 CreateTables.sql . . . 72

4 Testing **74**
4.1 Video . . . 78
4.2 Testing plan . . . 78

5 Evaluation **82**
5.1 Objective evaluation 82
5.2 Third-party feedback 82
5.3 Feedback evaluation 82
5.4 Improvements 82

1 Analysis

1.1 Problem statement

To form the perfect grouping for a group project within a school, many factors need to be taken into consideration to prevent students from feeling incapable of completing the project. These factors include, but are not limited to, skills of the students, skills required by the project, and number of students in the group.

This project aims to create a tool to aid teachers in ensuring the students are separated into groups where the students are best able to get along and combine their individual skills to achieve their project's aims. The user can input the data that they want to be taken into consideration by the program and it will output a set of groups. This is in the form of a table on a GUI - to present the results in a readable way - and an option to export the data.

This program will not find, assume, or generate information about the students or project, it will only work with the data inputted to assign a group of students to a project.

Teachers will benefit from this tool as they don't have to spend as much time or effort creating groups for the projects they set for their students, so they can spend their time and effort on helping and teaching the students instead. Also, when the students are working on their projects, they are more likely to be focused on their work, so require less oversight from the teacher.

Students will also benefit from this tool as they would be able to apply their skills to a project in a group of people they can enjoy time with, rather than being stuck on a project they have no interest in with people they do not get along with. This means they are more likely to enjoy their time at school, be able to get more work done, and therefore be able to learn more, as opposed to being unmotivated and distracted in a poorly made group.

1.2 Research

1.2.1 Sources of inspiration

1. Random group generator from ClassTools.net [Fig. 1]
   https://www.classtools.net/random-group-generator/

   - This tool is used frequently by my teachers to create random groups for group activities.

- The GUI is very simple to use as there is little data that needs to be entered, so can all fit down the left of the screen.
- The output is very easy to read as each group is visually separated using both coloured boxes and numbering. [Obj. 3aii, 3bi, 5c]
- However, this tool is too simple because it lacks any way to group with conditions, which is part of my scope, as they are purely randomised. This means that teams could quit often be unfairly balanced in terms of skill, or group students together that are known to not work well together. The teacher's only option to solve this is to re-roll until the output is suitable enough for the task manually. This gave me the idea for the algorithm to be the simulation of this re-rolling and selection of optimal outputs.

2. keamk [Fig. 2] [Fig. 3]
   https://www.keamk.com/random-team-generator
   - This tool creates groups that all have a balanced level of skill.
   - I like how the skill level is inputted as a rating between 1-5 as this is intuitive for the user.
   - I also like the extra details that are outputted, such as how many are in each group next to the group name and the skill levels next to each individual. [Obj. 3a]
   - Furthermore, the option to go back to make small changes to the data inputted and get new groupings was very quick and useful. [Obj. 1h]
   - However, this tool can only create balanced groups, not groups with different objectives as is in my scope, and there is no way to assign a certain number of participants to each group, as the program puts the same number (or one away) of participants in each group. [Obj. 1f, 2c, 2d]

These tools influenced me to reduce the complexity of my GUI, and gave me ideas such as to keep groups visually separate using colour as well as tables, adding the option to go back and edit the inputted data later, and some summary statistics to include with the outputs.

1.2.2 Algorithm

I have researched into representing students and projects into bipartite graphs, and then using matching algorithms such as the alternate path algorithm to join students to projects [Fig. 4]. However, this will only match one student to one project, not groups so is not suitable. I then researched applying the algorithm repeatedly to graphs with randomised students, but this means that some students will never be joined into a group purely based on chance and that the groups formed are such that each individual is best for the project, not that the overall group is best suited for the project, so teamwork will not

as good as it could be. I then researched using a grouping algorithm, such as https://help.feedbackfruits.com/hc/en-us/articles/23527084211346-Group-Formation-Algorithm-Explainer to create similar groups then using bipartite matching algorithms to match them. However this only creates groups with similar skill sets, not skill sets that vary based on the project.

I decided on a genetic algorithm where each individual is made up of a possible solution and the fitness score is an average between how suited each group is to its assigned project [Fig. 6]. The individuals with higher fitness scores are discarded and the lower ones are randomly selected to crossover to form new individuals. Some of these individuals formed may contain error such as the same student multiple times, or a missing student, so that must be corrected before continuing. Then, random mutations are applied to some individuals, which may involve swapping of students between groups, moving some students to an optional project, or moving some students to group that isn't full. New fitness scores are calculated and the process repeats a predetermined number of times [Fig. 9]. I chose this because genetic algorithms are very adaptable to varying conditions and constraints, such as the ones that the user chooses in this program. This will allow the program to test different combinations of projects and students, test different group sizes for a project, and swap in and out optional projects to find better and better solutions until it reaches a high-quality solution that fits all the user's needs. This fits the project scope far better than all previously researched algorithms.

Also, the runtime of a genetic algorithm can be changed without changing the input data, such as by changing how many iterations it undergoes. I found this helpful in testing, especially when trying to group large numbers of students. [Obj. 1g, 6b]

1.2.3 Inputting data

For the inputting of data, I have decided that it can be inputted through txt and csv files [Obj. 1b], manually on the GUI [Obj. 1a], or any combination of those methods [Obj. 1d]. This is because the data for different individuals and different projects could be stored in different ways within the same system, and manual tweaks may be needed by the user to ensure all data is as is needed by the application. I will also include the option to import the data from an Excel spreadsheet at the request of my third party [Obj. 1biv].

1.2.4 Accounts

I have decided against implementing an account system because I don't think it will provide any value to the program. Accounts are typically used for ensuring data can only be accessed by the users with the correct permissions. As this program will be designed to be run locally, all the data will be stored and only accessible on the machine the program was run on, so the OS can handle user security.

Another reason for accounts is allow users to access their data that has been stored by a third party, especially within a cloud to allow for data to be quickly transferred between devices. I will implement a manual solution to this by allowing users to export and import all the data the program stores and the user can handle that data transfer themselves [Obj. 1b, 4c, 4d].

1.2.5 Internal data storage

I chose to store the data about the students and projects in their own classes, that are aggregated within an 'individual' class. The student and project classes contain their names, skills and the project will have the recommended group size. The individual will contain a project object, student objects, and current fitness score as well as methods for calculating fitness scores, crossover with other individuals, and mutations. [Fig. 7]

This keeps the attributes and methods stored together in the program to make the code easier to read and understand by a programmer.

1.2.6 External data storage

[Obj. 4a, 4b] I have researched into whether information about projects and students should be stored somewhere, and if so, how. I have concluded that they should be stored in relational databases that are accessible and modifiable by entering the details into the graphical interface, which are passed as parameters into SQL statements. These can then be loaded as a list of checkboxes that can be selected and deselected by the user for that run. [Obj. 1c]

I chose to store the data because being able to make minor changes to the data and then re-running the program quickly is a very valuable feature that could save users' time and energy as they would not have to go through the entire process of entering every piece of data just for small changes to some of the data. This feature is present in keamk's grouping tool, and I found it very useful in my testing.

I chose databases to store the data because I have more experience working with relational databases and SQL then any other form of data storage, such as in files. The data is structured so databases would allow for efficient access and modification of specific entries.

1.2.7 GUI library

Between the two main python GUI libraries, pygame and Tkinter, I chose Tkinter because of the larger library of classes of pre-made, customisable widgets that can be used more easily to produce a simple, easy-to-use tool, such as a dialog that allows users to graphically navigate through their files to find their data file [Fig.5].

On the other hand, with my prior experience with pygame, I have found it to be more suited towards building games and graphically-complex tools, which is not what I am aiming for in this project. I am aiming to have a GUI

that is quick and simple to create and edit, and has user-friendly features for inputting, editing, and reading data. I also have more experience with Tkinter, which will reduce the amount of time I spend on the GUI and allow me to focus on developing and improving the grouping algorithm and user experience.

1.3 Dialogue

My third party is Dr Ed Horncastle, the lead teacher for the Exeter Mathematics Certificate (EMC) at my school, which is where groups of 3 or 4 students work together on research projects set by University academics or companies. He is suitable because he has to make these groups based on the skills of the students and match them to projects every year, so he has a thorough understanding of how this tool could work and would benefit from using this tool.

1.3.1 Initial interview

The following questions were asked during the research stage:

1. What is the current process for creating EMC groups?
    - All the metrics to consider are gathered into a spreadsheet and then groups are formed by hand.
    - Some projects don't end up getting groups assigned, especially those that have been set by a company that already has another project with an assigned group.
2. What are these metrics?
    - Students' own choice
    - There must be a mix of genders
    - The projects must be relevant to the A-level studies of that student. Someone who does not take computer science shouldn't be given a computer science based project
    - Those that work hard and those that laze off more easily should be mixed together. If this is their first project, this is inferred from GCSE results, otherwise it is based off the results of previous projects.
    - Special educational needs
    - Timetabling restrictions
    - Students do not get put into groups with others they have worked on projects with before
3. How are the student choices measured?
    - For some of the projects, the students give their top 3 choices of projects. Starting with the least popular project, the groups are formed by trying to minimise the sum of the positions the student's ranked the project in.

4. How is the data about the students and projects currently stored?
    - Excel spreadsheet
5. How would you prefer to input the data?
    - Student choices from a Microsoft Form and all other data from a Excel spreadsheet
6. How much data and in what format do you think the data should be outputted?
    - In tables with the project name, student names and summaries of information about each group such as gender ratio, how many take computer science or physics, and how many have special education needs.
    - As a simplified register in alphabetical order
7. How useful would storing project/student details locally, that can be loaded when needed, be for you?
    - Very
8. Are there any other features that would be useful for this tool to include?
    - Some extra features that could be added is timetabling for what order the projects are presented in, and itinerary for group trips out of school.

After this, I decided to include file imports as an input method [Obj. 1b], include options for projects to not have groups assigned despite being a part of the algorithm [Obj. 1f], take genders into account [Obj. 2e], have summary statistics presented alongside the groups [Obj. 3a], have the option to view the groups without any extra information [Obj. 3b], store the data locally for re-use at another time [Obj. 4].

I decided not to include student choice, student personality, special education needs, timetabling restrictions, or preventing grouping the same students together across multiple as adding too many restrictions to the algorithm will make it take it longer to run to create a set of groups that best matches all of these restrictions. These are the restrictions that were lower on Dr Horncastle's priorities. I will also not include timetabling or scheduling as these deviate quite far from my project's scope.

1.3.2 Follow-up questions

The following questions were asked during the designing stage:

1. I have decided on using three factors to determine how "good" a group is: the number of students with at least one relevant skill to the tasked project; the number of skills required by the project that the group meets; whether or not there are at least two different genders in the group. Could you rate how important these factors are in your opinion, or whether you think any are equal?

    - The students being able to contribute is twice as important as the other two factors combined
    - Project skills are slightly more important than gender

1.4 Objectives

Objectives beginning with a * are extra objectives, only implemented if time allowed and are not required for the program to meet the scope of the problem.

1. Data can be entered into the program
    (a) Data can be entered manually into the GUI
    (b) Student and project data can be entered through file import
        i. File type can be csv
        ii. *File type can be txt
        iii. *File type can be a SQLite3 database
        iv. *File type can be xlsx
    (c) Data can be loaded from stores from previous uses of the program
    (d) Data can be entered through a combination of other methods
    (e) Student data must contain a name, gender, and skills
    (f) Project data must contain a name, maximum group size, minimum group size, whether or not the project must have a group or not, and skill requirements
        i. Maximum group size must be greater than 0 and greater than minimum group size
        ii. Minimum group size must be greater than 0
    (g) Settings data must contain a target fitness, max number of iterations, and the size of the populations
        i. Target fitness must be between 0 and 1
        ii. Max iterations must be a positive integer, not including 0
        iii. Size of populations must be a positive integer greater than or equal to 4
    (h) Project and student data can be edited and removed
    (i) At least one student, at least one project, target fitness, max iterations, and size of populations must all be entered/have values before the algorithm is run

2. Students must be grouped together and to a project
    (a) All students must be put into one group each
    (b) *All students must have at least one skill that will allow them to contribute to the project, when possible
    (c) Projects must not have more students assigned to it than the maximum limit inputted by the user
    (d) Projects must not have less students assigned to it than the minimum limit inputted by the user
    (e) *Groups must have a mix of genders, when possible
3. The resulting groups must be viewable
    (a) The user must have to option of coloured tables displayed on the GUI
        i. These tables contain the project name, student names, number of students, gender ratio, and skill range
        ii. Adjacent tables are coloured differently from each other
    (b) The user must have the option of a register layout displayed on the GUI
        i. Each project and its details are stored in separate boxes with a background distinct from its adjacent neighbours
        ii. Each box contains the project name, and the names of students in that group
        iii. Projects are displayed in alphabetical order
        iv. The students in each group are displayed in alphabetical order
    (c) *Both above options can be exported to locally stored files
        i. *File type can be csv
        ii. *File type can be txt
        iii. *File type can be xlsx
4. All data must be stored
    (a) The data about projects and students must be stored in relational databases
    (b) *The data about results must be stored in relational databases
    (c) The user must have the option to export the databases' contents to locally stored files
        i. File type can be csv
        ii. *File type can be txt
        iii. *File type can be xlsx
    (d) *The settings data must be stored as a JSON file
5. The design must be consistent across all pages

(a) Text is black with the Arial font

(b) The top of each page that displays data must feature a header

   i. The header contains text that provides a title for the data shown

   ii. The title text must be displayed on the left with a significantly larger font to the data shown

   iii. The header colour must be of hex code #FFDB32

   iv. The header may contain buttons, solely for navigation, on the right

(c) Data about different entities must be visually separate by keeping all the data about each single entity within individual boxes

(d) Buttons that perform a function that cannot be performed at that time must be greyed out, and feature text below stating how to enable the button

(e) Buttons that lead to a different page must feature the name of the page, or the relationship between the current page and next (e.g. "back")

(f) Up to 6 colours can be used to visually separate entities through colour. Red, orange, yellow, green, blue, purple. These colours must have RGB values all greater than 186 to ensure the text remains legible

6. Users must be happy with the program

(a) 100% of users must be happy with navigating the program

(b) 100% of users must be happy with response times

(c) 100% of users must consider the groups outputted by the program to be suitable based on the provided data

(d) 100% of users must consider the methods of data entry to be convenient

(e) 100% of users must consider the methods of exporting data to be simple

(f) 100% of users must be able to read all data shown on the GUI

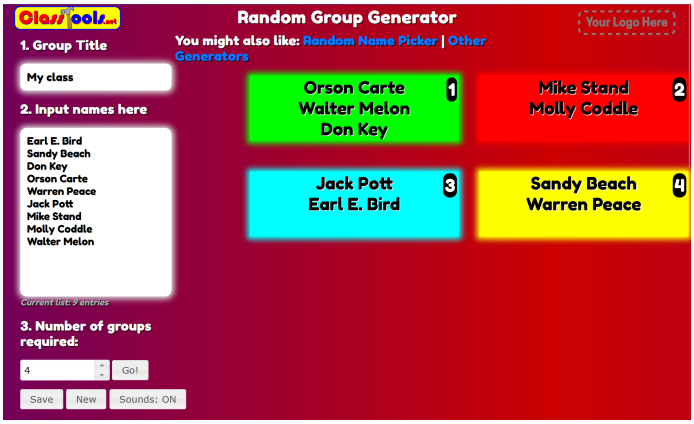


Figure 1: ClassTools's random group generator tool with inputs and outputs visible on the same page

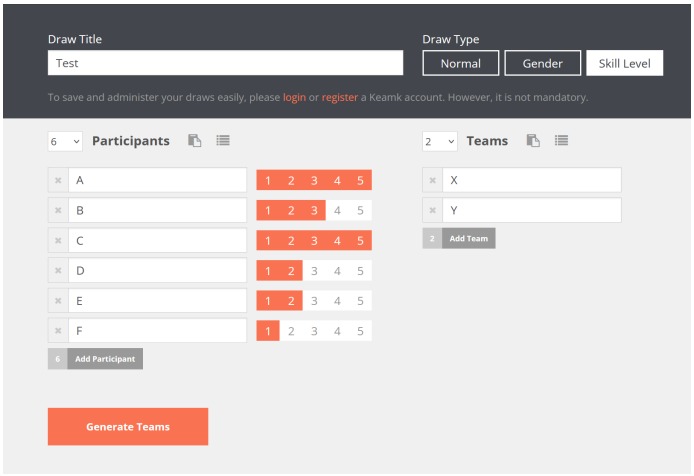


Figure 2: Keamk's menu for inputting data



Figure 3: Keamk's outputs

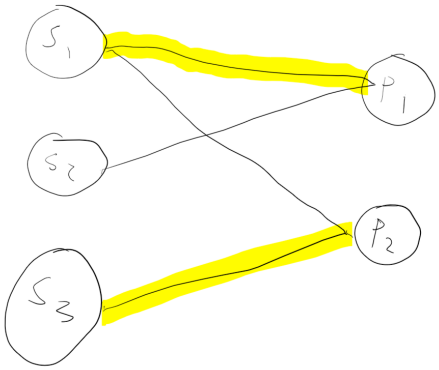


Figure 4: A bipartite graph with 3 student nodes on the left, 2 project nodes and the right, and some edges between them. A matching algorithm has been applied, with the matching highlighted in yellow. This has not met the scope because a student has not been matched.

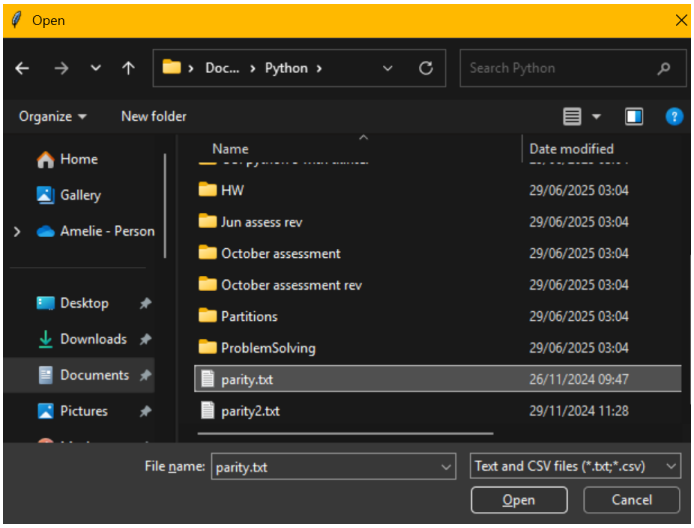


Figure 5: Tkinter's file dialog, configured to only show .txt and .csv files

1.5 Modelling

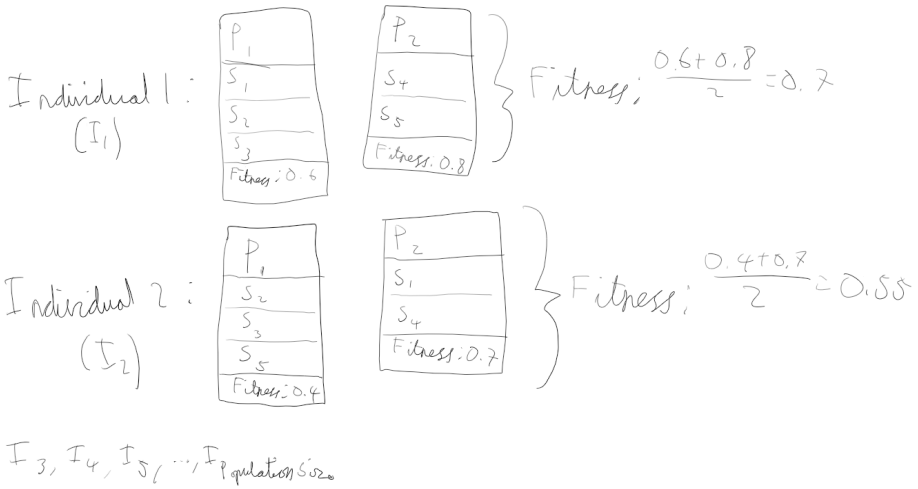


Figure 6: A sketch showing an example of a population, representing a set of possible solutions (individuals). Each individual represents a possible set of groups, and has a fitness score that represents how well every project's group matches the project's and user's requirements. This is calculated by averaging all the fitness scores of the groups it contains.

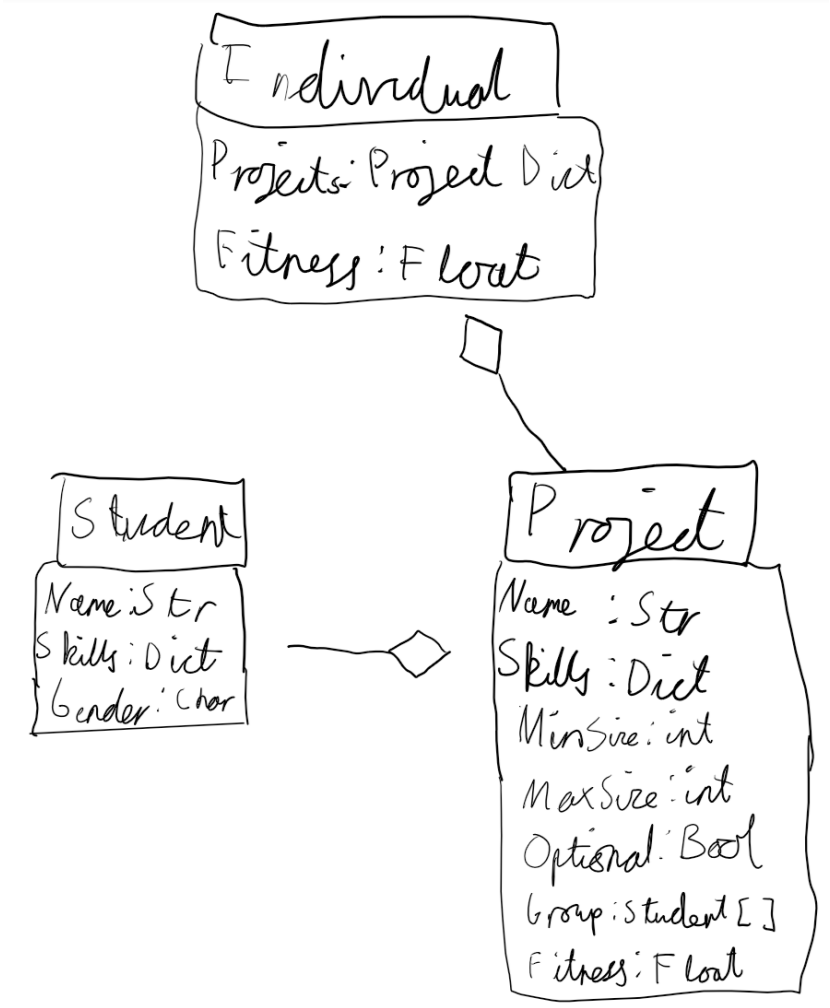


Figure 7: Class diagram showing relationships between the classes and their attributes used by the genetic algorithm.

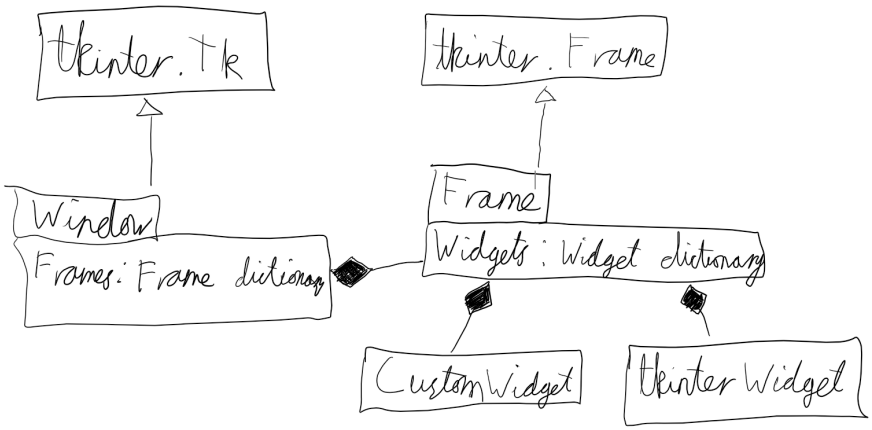


Figure 8: Class diagram showing the general relationships between the types of classes that form the GUI

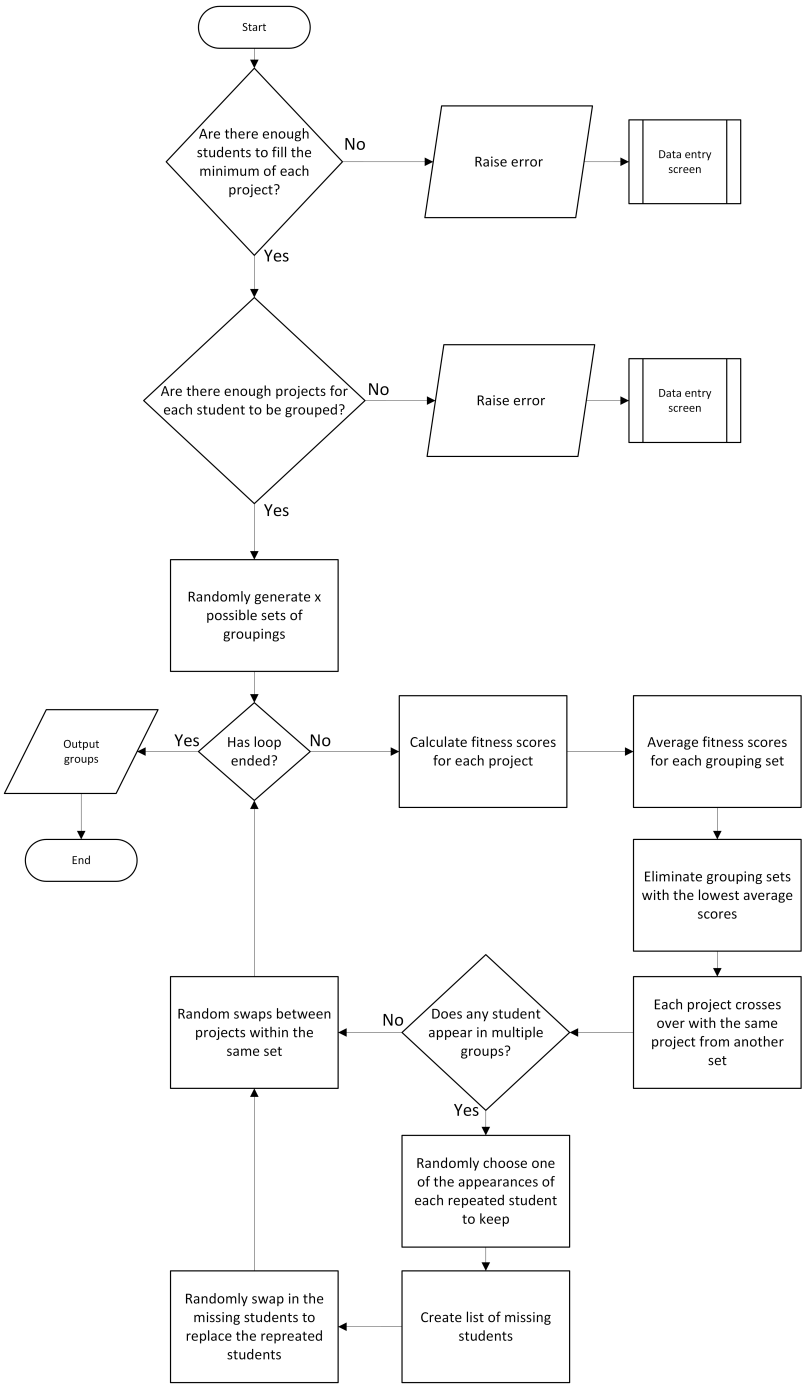


Figure 9: Flowchart showing a rough outline for the genetic algorithm

2 Design

2.1 GUI

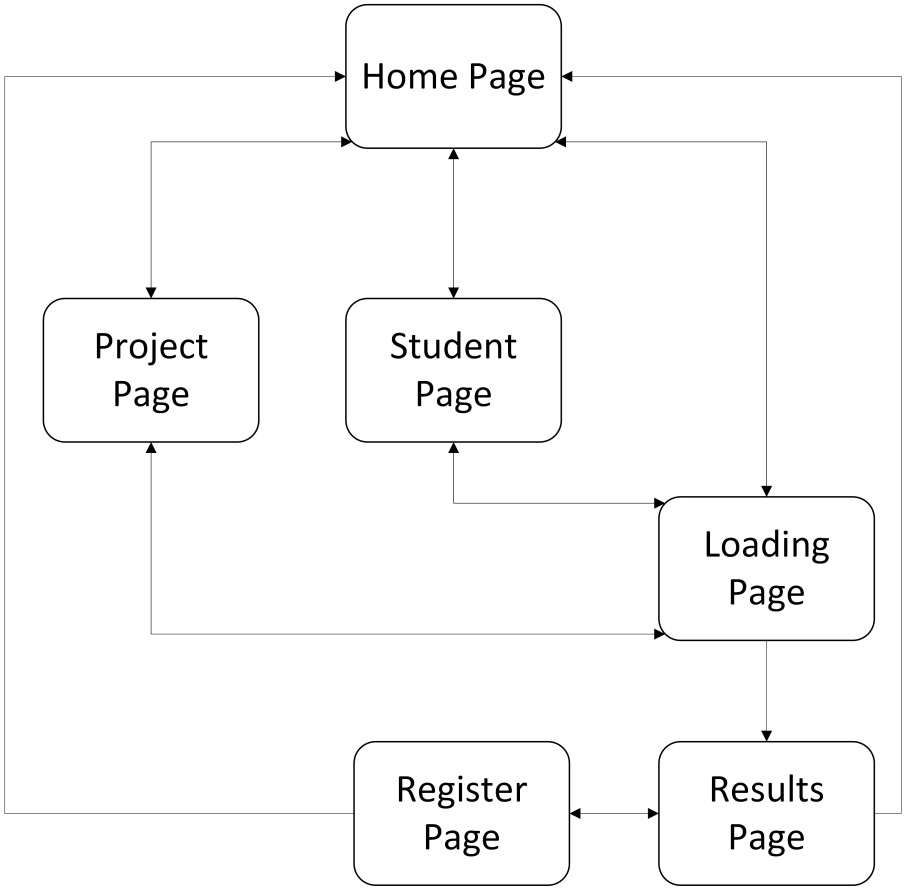


Figure 10: Page navigation diagram showing which pages can navigate to each other. In the technical solution, the links between the student and project pages, and the loading page are not present in order to prevent user from accidentally cancelling the genetic algorithm.

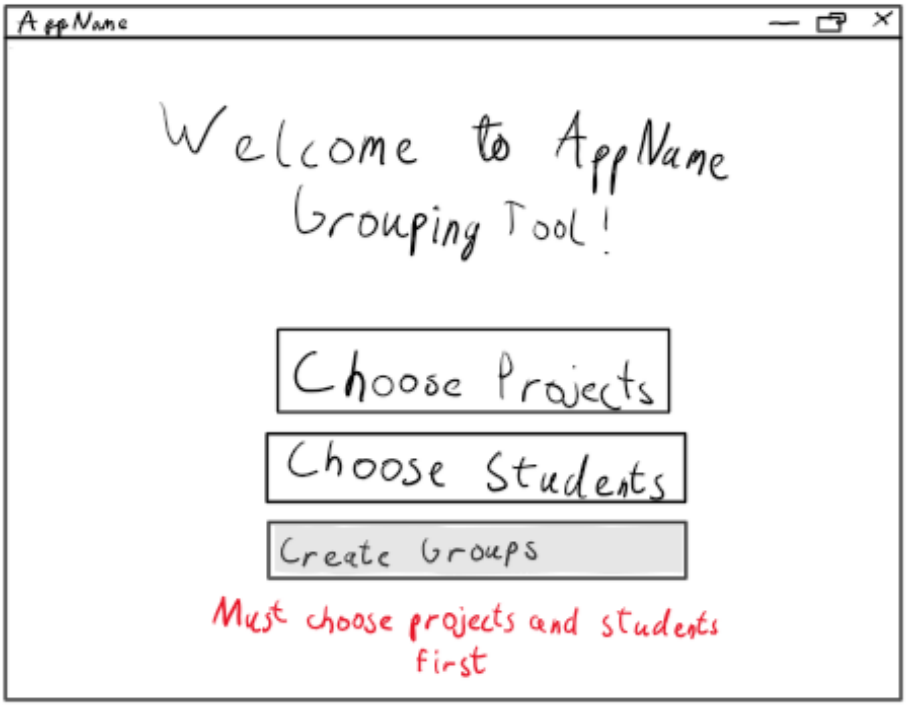


Figure 11: Sketch of the homepage layout that is the first page seen upon opening the app. The "Choose Projects" and "Choose Students" lead to the projects and students pages, respectively. The "Create Groups" button leads to the loading page and runs the grouping algorithm with the students and projects the user has selected. It is greyed out and not clickable unless at least one project and student has been selected [Obj. 1i]. The technical solution also features a button at the top of the page that leads to the settings page

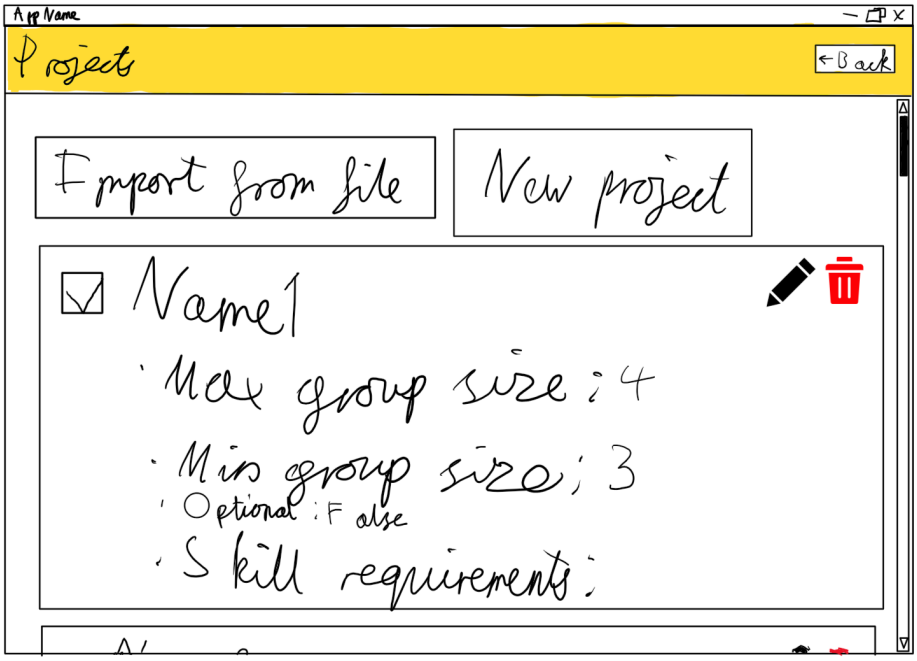


Figure 12: Sketch of the project page layout. The projects in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import project data from supported files [Obj. 1b] and to manually enter project details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The project details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each project in the list features a checkbox, allowing users to select which of the projects in the database will be used in the grouping algorithm. Projects not selected will not have groups matched to them. This page also features a back button to go back to the home page.

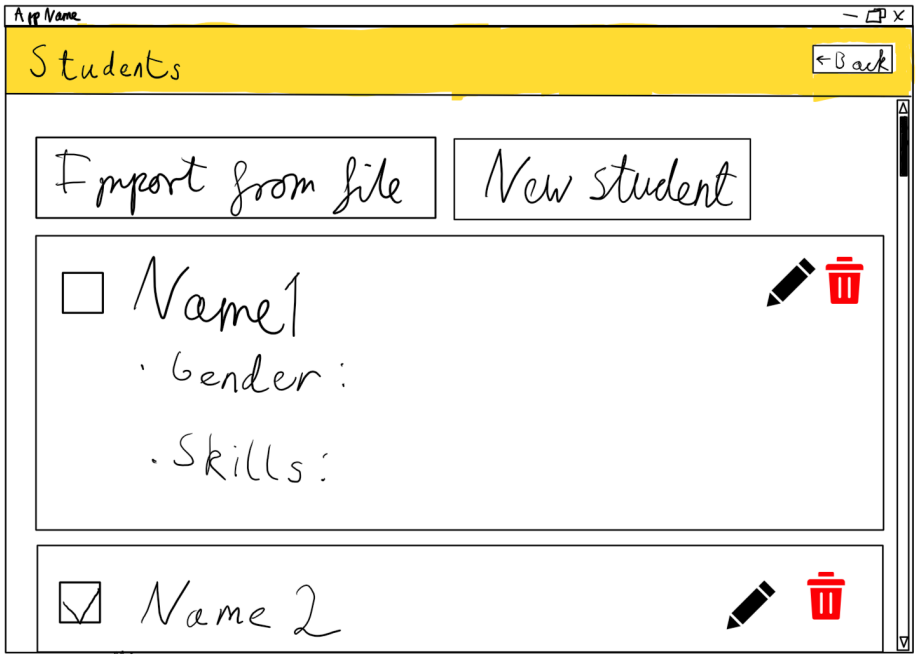


Figure 13: Sketch of the student page layout. The students in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import student data from supported files [Obj. 1b] and to manually enter student details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The student details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each student in the list features a checkbox, allowing users to select which of the students in the database will be used in the grouping algorithm. Students not selected will not be put into groups. This page also features a back button to go back to the home page.

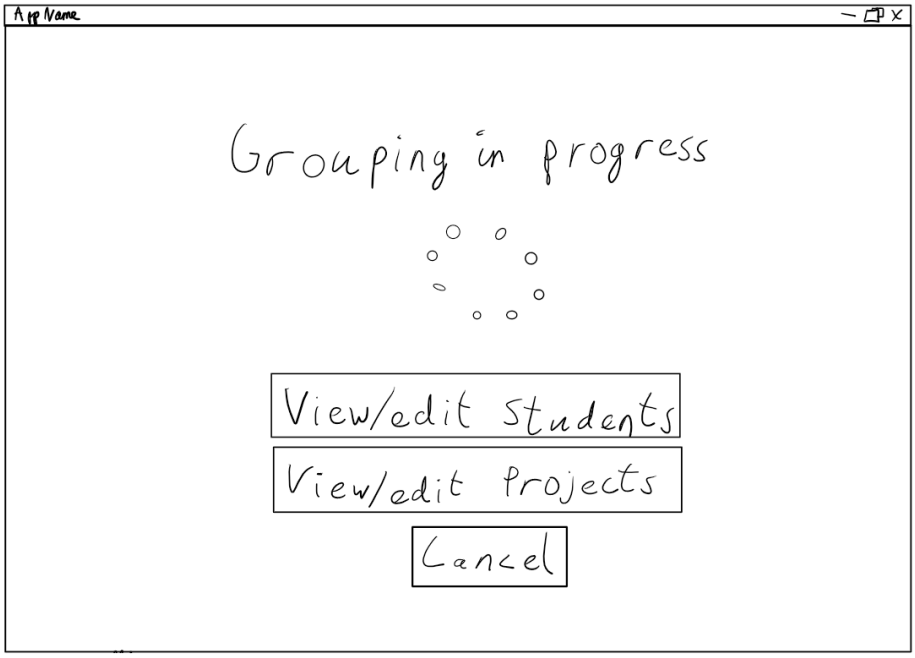


Figure 14: Sketch of the loading page layout. This page features a loading animation, and three buttons. The "View/edit Students" and "View/edit Projects" buttons lead to the Students and Projects pages, respectively [Obj. 1g]. The "Cancel" button ends the algorithm and leads to the home page. Once the algorithm has finished successfully, it moves onto the results page. In the technical solution, the edit students and edit projects buttons are not present because any changes to the input data would mean cancelling the genetic algorithm anyway, so users instead only have the option of cancelling the algorithm to make this clearer.

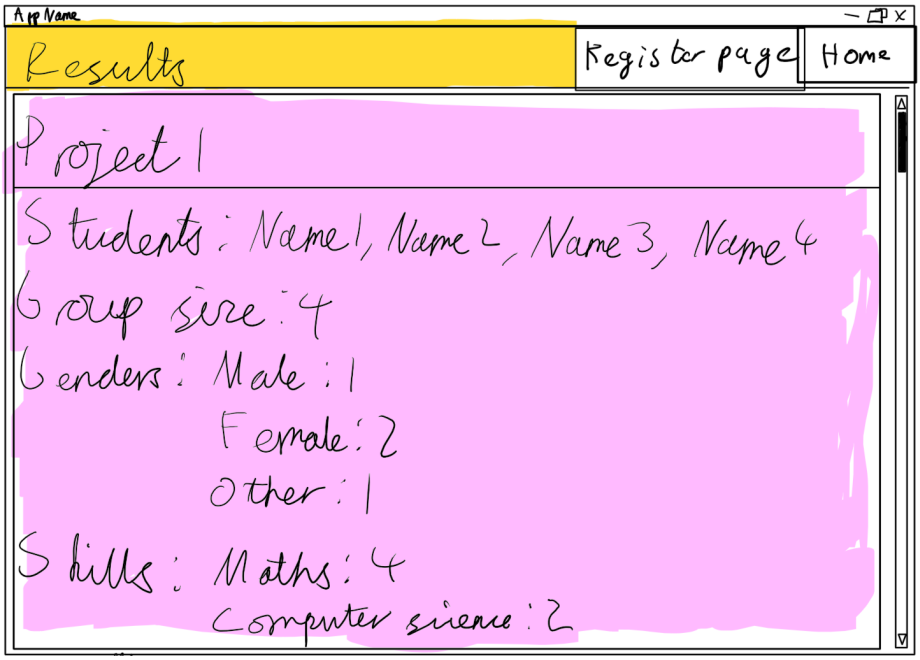


Figure 15: Sketch of the results page layout. It features a scrollable list of all the projects with the students grouped to it and summary statistics about the group, such as group size, gender ratio, and skills [Obj. 3a]. The "Switch view" leads to the register page and the home button leads to the home page.

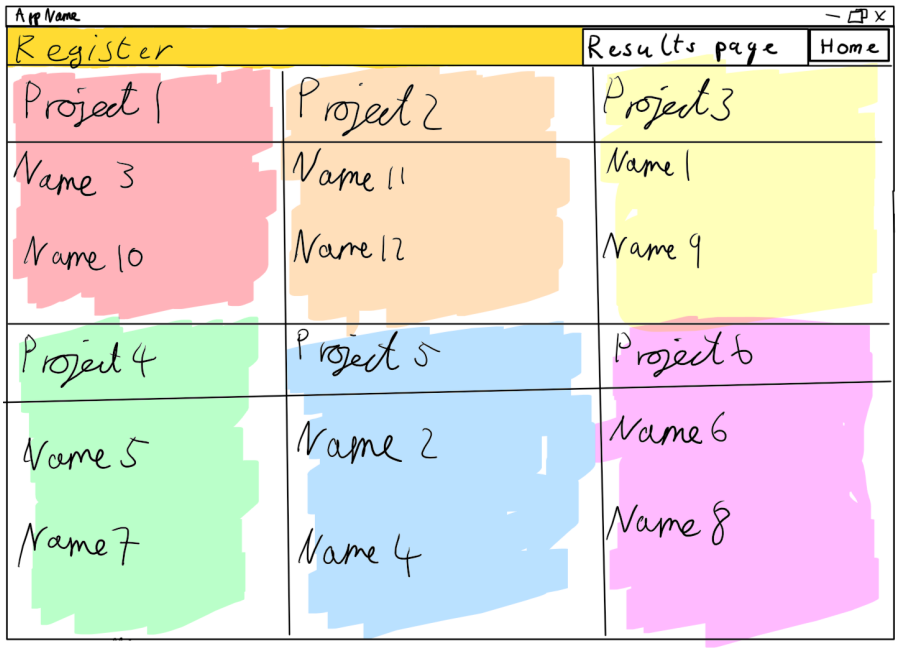


Figure 16: Sketch of the register page layout. It features the projects arranged in boxes in a grid layout with only the project name and student names. The projects are arranged in alphabetical order, and the students within each group are arranged in alphabetical order.

2.2 Databases

- Students(StudentID, Name, Gender)
- StudentSkills(StudentID, Skill)
- Projects(ProjectID, Name, MaxSize, MinSize, Optional)
- ProjectSkills(ProjectID, Skill)
- Results(ResultID, ProjectID, LocalTimeGenerated, EpochTimeGenerated)
- ResultStudentLink(ResultID, StudentID)

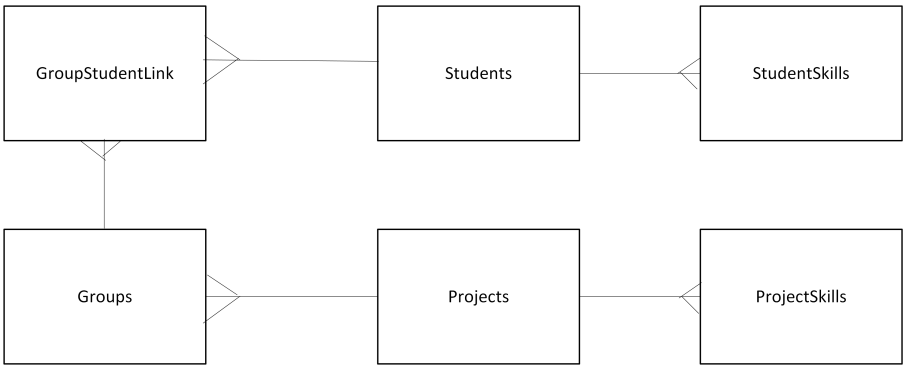


Figure 17: An entity relationship diagram showing the one-to-many relationships between the Students and Projects entities and the store of their skills to ensure all fields are atomic so the database is normalised.

2.3 Inputs and outputs

Table 1: An IPSO chart showing what data is inputted, how it is processed and stored, and how the results are outputted.

Input	Process	Storage	Output
Project details: - Name - Max group size - Min group size - Optional - Skill requirements	Validate input	Projects and ProjectSkills tables in database	Display on GUI [Fig. 12]

Student details: - Name - Gender - Skills	Validate input	Student and StudentSkills tables in database	Display on GUI [Fig. 13]
Settings: - Directory to export database to - Target fitness for genetic algorithm - Max iterations for genetic algorithm - Size of populations for genetic algorithm	Validate input	JSON file	NA
"Create Groups" button on homepage pressed	Run genetic algorithm[List. 1]	Classes for Student, Project, and Individual [Fig. 7]	Scrollable list of projects and groups [Fig. 15]

Table 2: Validations and restrictions on data entry

Data	Data Type	Validation and Restrictions
Projects		
Name	String	- Required
Max group size	Integer	- Required - Positive integers only
Min group size	Integer	- Required - Positive integers only - Must be less than or equal to 'Max group size'
Optional	Boolean	- Lack of entry is equivalent to entry of false
Skill requirements	String	- Required - Entries separated by commas
Students		
Name	String	- Required
Gender	String	- Required
Skills	String	- Required - Entries separated by commas
Settings		

Directory to export database	String	- Must be a valid directory - Program must have read-/write permissions in that directory
Target fitness for genetic algorithm	Float	- Default is 0.9 - Must be between 0 and 1 inclusive
Max iterations for genetic algorithm	Integer	- Default is 50 - Must be a positive integer not including 0
Size of populations for genetic algorithm	Integer	- Default is 100 - Must be a positive integer greater than or equal to 4

2.4 Algorithm

2.4.1 Pseudocode

Listing 1: Pseudocode for the function that runs when on the loading page. It ensures it is possible to find groupings with the given requirements, then runs a genetic algorithm to find a good set of groupings. Some subroutines have flowcharts below. The mutate method is unused in the technical solution as the crossover method produces suffucient randomness so designing and coding the mutate method would be a waste of time.

```
1  TARGET_FITNESS = integer
2  MAX_ITERATIONS = integer
3  POPULATION_SIZE = integer
4
5
6  students = set of 'student' objects
7  projects = set of 'project' objects
8
9
10 #Ensure there aren't too many or too few students to meet
       ↪ requirements
11 minStudents = 0
12 maxStudents = 0
13
14 REPEAT for all projects:
15     maxStudents += project.maxSize
16     IF (not project.optional):
17         minStudents += project.minSize
18
```

```
19 #Check for too few students
20 IF (length of students < minStudents):
21     DisplayError("Error: Only {length of students} to
       ↪ fill a minimum of {minStudents} places.")
22     HomePage()
23
24 #Check for too many students
25 ELSE IF (length of students > maxStudents): #Checks for
   ↪ too many
26     DisplayError("{length of students} is too many to fit
       ↪  in {maxStudents} places.")
27     HomePage()
28
29
30 #Generate initial grouping sets and average fitness
   ↪ scores
31 population = list
32 REPEAT for i = 0 to POPULATION_SIZE - 1:
33     groups = GenInitialGroups(projects, students)
34     avgFit = (SUM fitness of each group)/(length of
       ↪ groups)
35     APPEND groups,avgFit to population
36
37 SORT population by avgFit
38
39 #Genetic algorithm
40 REPEAT for i = 1 to MAX_ITERATIONS:
41     #Ends early when one set of groupings is good enough
42     IF (highest avgFit > TARGET_FITNESS):
43         BREAK
44
45     #Eliminate the worst half of grouping sets
46     population = first (POPULATION_SIZE FLOORDIV 2) of
       ↪ population
47     new_pop = []
48
49     #Create next population
50     REPEAT for j = 1 to 90% of POPULATION_SIZE:
51         #Crossover random groupings
52         groups = Crossover(random group from population,
               ↪ random group from population, students)
53
54         #Ensure all students appear once
55         valid = verifyGroupings(groups)
56         IF (not valid):
57             fixGroupings(groups)
```

```
58
59          #Random mutations to prevent convergence towards
                ↪ a local best solution
60          Mutate(groups)
61
62          #Find the average fitness score for each new
                ↪ grouping
63          avgFit = (SUM fitness of each group)/(length of
                ↪ groups)
64          APPEND groups,avgFit to new_pop
65
66      #Combine the crossover with 10% elitism
67      population = new_pop + top 10% of population
68      SORT population by avgFit
69
70
71  return group with highest avgFit
```

2.4.2 Fitness algorithm

The fitness algorithm takes in the data about the current set of groupings and outputs a value between 0-1 that represents the suitability of the students to each project (0 being the worst possible groupings, 1 being absolutely perfect). This algorithm needs to allow the program to continually converge towards 1, guiding the program in the correct direction and helping it to reject bad solutions.

The factors that need to be considered by the algorithm are:

1. Each student should be able to contribute to their project [Obj. 2b] – they should have at least one skill that is demanded by the project
2. The demands of the project should be met by the students – the more skills are met by the students, the better the grouping
3. There should be a mix of genders [Obj. 2e] – there must be at least two different genders

These factors are of different levels of importance, so their contribution to the fitness value needs to be weighted. After talking with my third party, we decided on weights of 0.6, 0.25, and 0.15 respectively.

The fitness value for each group is then averaged to form a fitness for the whole set of groupings.

This can be seen in the pseudocode below [Listing 2]

Listing 2: Pseudocode for the above fitness algorithm. The technical solution differs from this slightly to account for division by 0 and floating point errors.

```
1  fitness = 0
```

```
2
3  REPEAT for all projects in individual:
4      skillsMet = set() #Records the skills the students
           ↪ have that the project demands
5      genderCritMet = False #Records whether or not there
           ↪ are multiple genders in the group
6
7      REPEAT for all students in project:
8          #Gets set of skills in common between the student
               ↪  and the project
9          commonSkills = intersection of student skills and
               ↪  project skills
10         #If student has at least one applicable skill,
               ↪ contribute to the fitness with a weight of
               ↪ 0.6
11         IF size of commonSkills >= 1:
12             fitness += 0.6/(number of students in project
                   ↪ 's group)
13
14             skillsMet = union of skillsMet and
                   ↪ commonSkills
15
16         #There will be at least two genders as soon as
               ↪ one student's gender is different from the
               ↪ first
17         #If this happens, add the weight of 0.15
18         IF not genderCritMet and this student's gender !=
               ↪  first student's gender:
19             genderCritMet = True
20             fitness += 0.15
21
22     #The more of the project's skills met by the students
           ↪ , the more of the 0.25 weighting can be added
23     fitness += 0.25*(size of skillsMet / size of project'
           ↪ s skills)
24
25 #Final fitness score for the individual
26 fitness = fitness / number of projects in individual
```

2.4.3 Diagrams

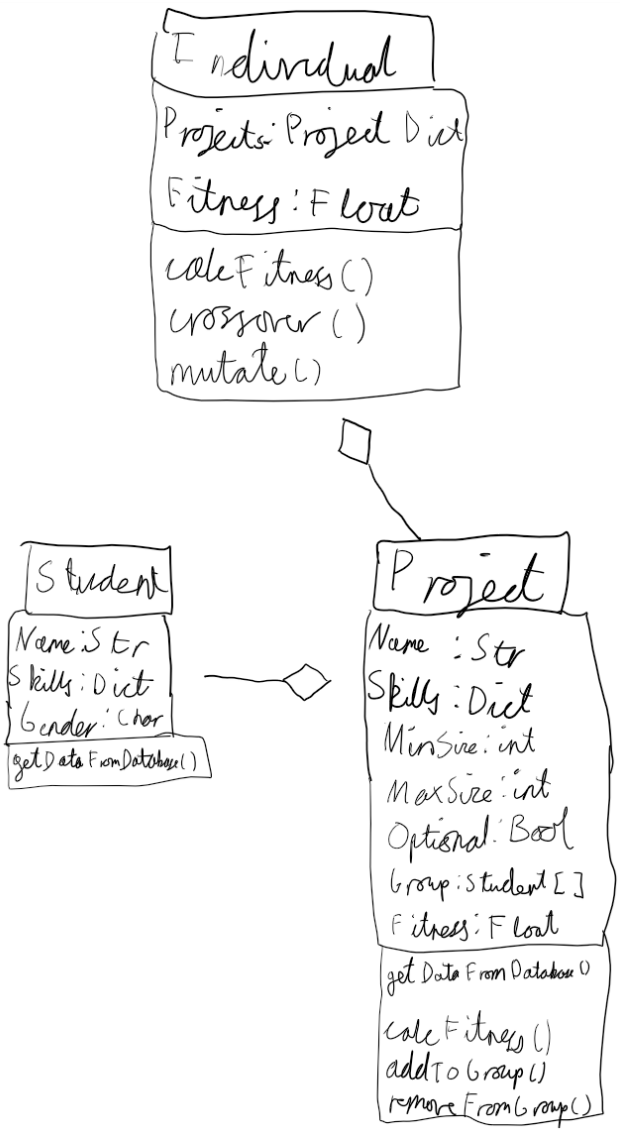


Figure 18: A class diagram showing the attributes and methods of the classes used by the genetic algorithm, and the relationships between each class. The technical solution differs as the database handling occurs in a subroutine, rather than a method so that projects can be sorted into "required" and "optional". Also, no students ever got removed from a project.

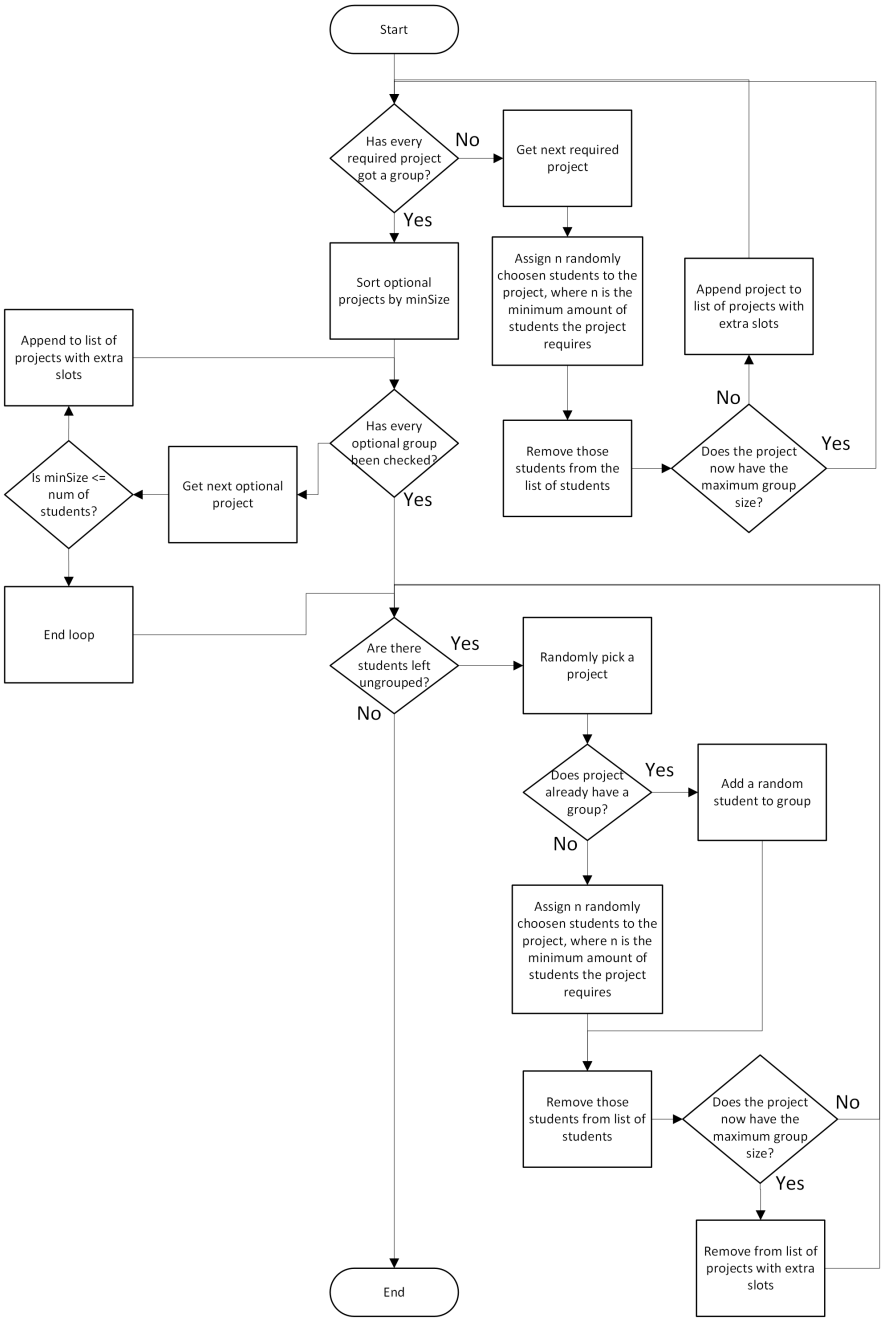


Figure 19: A flowchart for the GenInitialGroups subroutine on line 33 of the above pseudocode. It takes a list of all projects and a list of all students as parameters and returns an Individual object.

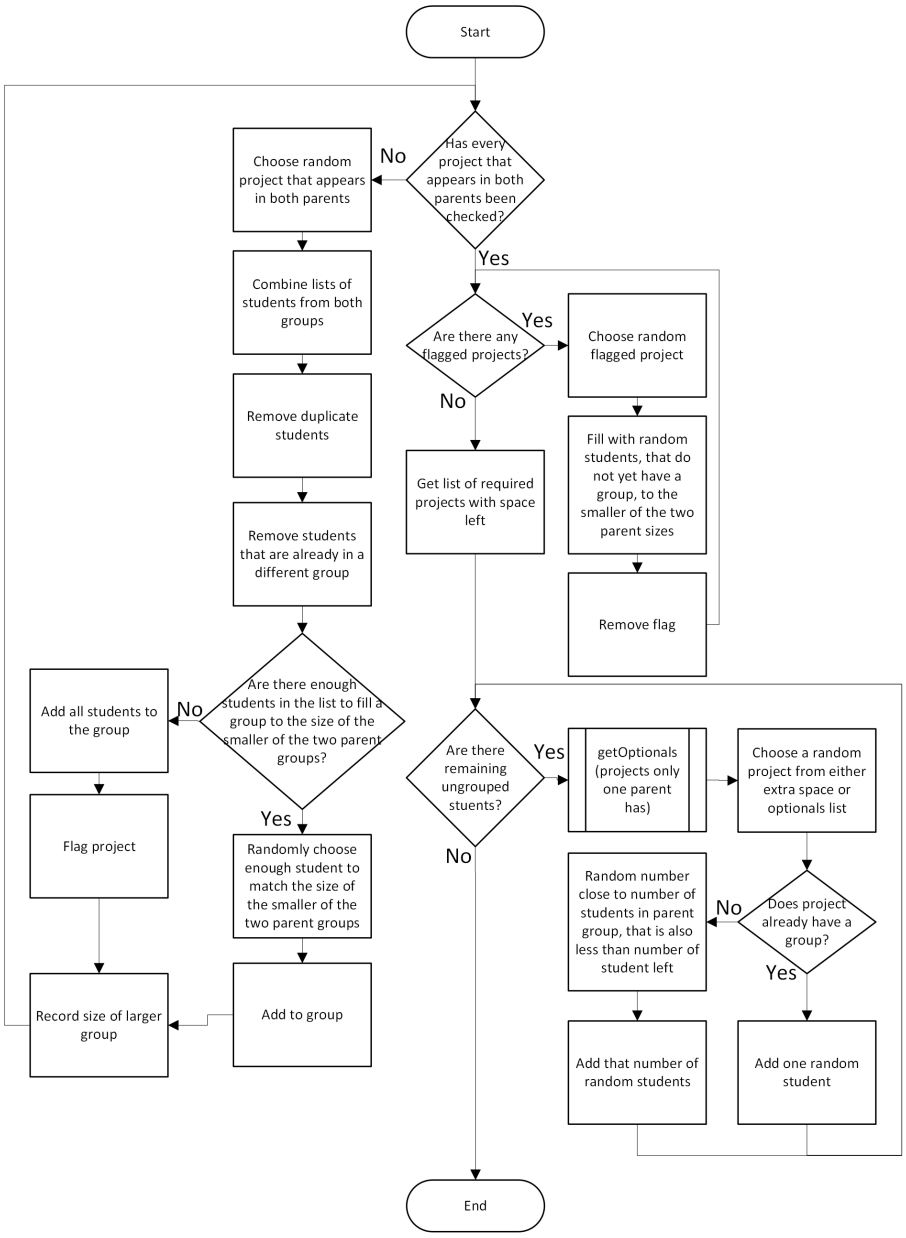


Figure 20: A flowchart for the Crossover method on line 52 of the above pseudocode and in the Individual class in the above class diagram. It takes two Individual objects, and the list of students, as parameters and returns an Individual object.

3 Technical solution

Contents

3.1 main.py 34
3.2 geneticAlgo.py 62
3.3 CreateTables.sql 72

All files should always stored in a folder called "Code", alongside a database.db file. main.py is the main file that is run by the user. It contains the GUI, and calls to the functions and classes in the other files when needed. All the pages outlined above [Fig. 10] have their own classes, whose objects are created dynamically and then stored in the window class. The page classes' attributes are mostly widgets, both tkinter widgets and custom widgets classes (such as Header, ScrollableFrame, SelectionBox, and ProjectBox), or the attributes store the result of user interaction, such as the state of the checkboxes on the project page. The methods all code for the processing and outputting the result of human interaction with the GUI.

geneticAlgo.py contains the code for genetic algorithm that calculates the groups. The subroutines follow the design of the above pseudocode [Listing 1] and the classes follow the design of the above class diagram [Fig. 18].

CreateTable.sql contains the SQL script for creating the database that the program uses to stores data.

Criteria

All line numbers are from main.py unless otherwise stated.

Criteria	Location
Group A Models	
Complex data model in database (eg several interlinked tables)	CreateTables.sql file
Complex user-defined use of OOP model	-Inheritance. E.g. projectBox inherits from selectionBox [Line 1011] -Composition e.g. ScrollableFrame in StudentPage [Line 448] -Polymorphism e.g. the tkraise method in HomePage [Line 88] -Abstract methods e.g. get_data in selectionBox [Line 1004]
Group A Algorithms	
Cross-table parameterised SQL	Line 147

Complex user-defined algorithms: optimisation	Genetic algorithm in geneticAlgo.py [Line 386]
Dynamic generation of objects based on complex user-defined use of OOP model	The show_frame method of the Window class [Line 51]
Group B Models	
Dictionaries	frames attribute in Window class [Line 38]
Text files	write_csv method of the SettingsPage class [Line 902]
Group B Algorithms	
Writing and reading from files	-Writing e.g. write_csv method of the SettingsPage class [Line 902] -Reading e.g. import_projects method of the ProjectPage class [Line 274]

3.1 main.py

```
1  import tkinter as tk
2  from tkinter import ttk, filedialog, messagebox
3  import sqlite3 as sql
4  import csv
5  import re
6  import geneticAlgo as ga
7
8
9  #=====GLOBAL CONSTANTS=====
10
11 BG_COLOUR="#FFFFFF"
12 HEADER_COLOUR="#FFDB32"
13 PASTEL_COLOURS=("#FFBABB","#FFDFBA","#FFFFBA","#BAFFC9","#BAE1FF","#
       ↪ FFBAFF")
14 LARGE_FONT=("Arial",30)
15 DATABASE = "database.db"
16 #SQLFILEHEADER="SQLite format 3"
17
18 #=====ROOT WINDOW=====
19 #Constants for styling included within
20
21 class Window(tk.Tk):
22     def __init__(self):
23         #Set up window
24         tk.Tk.__init__(self)
25
26         self.state('zoomed')
27         self.geometry(f"{self.winfo_screenwidth()-10}x{self.
       ↪ winfo_screenheight()}")
```

```
        self.title('Student Grouping Tool')
        self.config(background=BG_COLOUR)

        #=====STYLES=====

        HEADER_BUTTON_STYLE=ttk.Style()
        HEADER_BUTTON_STYLE.configure("Header.TButton",background=
    ↪ HEADER_COLOUR)

        #================
        #Dictionary that stores all loaded pages
        self.frames={}
        #Load the settings
        self.load_settings()
        #Load the database
        success=self.load_db()
        #Load the homepage first
        if success:
            self.show_frame(HomePage)

    #Method for changing currently open page
    def show_frame(self, f,*args,**kwargs):
        #If the page is already loaded, show it
        if f in self.frames.keys():
            frame = self.frames[f]
        #Otherwise, load it then show it
        else:
            frame = f(self)
            self.frames[f] = frame
            frame.place(relx=0,rely=0,relwidth=1,relheight=1)
        frame.tkraise(*args,**kwargs)

    def load_settings(self):
        if SettingsPage not in self.frames.keys():
            frame=SettingsPage(self,0.9,50,100)
            self.frames[SettingsPage] = frame
            frame.place(relx=0,rely=0,relwidth=1,relheight=1)

    def load_db(self):
        if not checkForDB(DATABASE):
            try:
                print("yes")
                queryFile=open("CreateTables.sql",'r')
                print("yes")
                query=queryFile.read()
                with sql.connect(DATABASE) as conn:
                    cur = conn.cursor()

                    cur.executescript(query)
                    conn.commit()
```

```
77                  return True
78              except:
79                  self.show_frame(BrokenPage,"Database could not be created
    ↪ .")
80                  return False
81          return True
82
83  #=====PAGES=====
84
85  class BrokenPage(tk.Frame):
86      def __init__(self,parent):
87          tk.Frame.__init__(self,parent)
88
89          ttk.Label(self,text="Something went wrong. Please restart the
    ↪ program.", font=LARGE_FONT).pack()
90
91          self.label=ttk.Label(self,text="Reason: Unknown")
92          self.label.pack()
93
94      def tkraise(self, reason="Unknown", *args, **kwargs):
95          self.label.configure(text=f"Reason: {reason}")
96
97
98
99  class HomePage(tk.Frame):
100     def __init__(self,parent):
101         #Set up home page layout
102         tk.Frame.__init__(self, parent)
103
104         self.settingsButton = ttk.Button(self,text="Settings",command=
    ↪ lambda:parent.show_frame(SettingsPage))
105         self.settingsButton.pack()
106
107         self.label = ttk.Label(self, text="Welcome to the Student
    ↪ Grouping Tool!", font=LARGE_FONT)
108         self.label.pack()
109
110         self.projectButton = ttk.Button(self,text="Choose Projects",
    ↪ command=lambda:parent.show_frame(ProjectPage))
111         self.projectButton.pack()
112
113         self.studentButton = ttk.Button(self,text="Choose Students",
    ↪ command=lambda:parent.show_frame(StudentPage))
114         self.studentButton.pack()
115
116         self.runButton = ttk.Button(self,text="Create Groups",command=
    ↪ self.get_data_for_algo,state="disabled")
117         self.runButton.pack()
118
```

```
119         self.warning = ttk.Label(self,text="Must choose projects and
    ↪ students first")
120 
121 
122     #Method override that runs once everytime the page is opened
123     def tkraise(self,*args,**kwargs):
124         tk.Frame.tkraise(self)
125 
126         #If at least one student and one project has been selected, allow
    ↪  the run button to be clickable and remove the warning message
127         if ProjectPage in self.master.frames and StudentPage in self.
    ↪ master.frames and self.master.frames[ProjectPage].
    ↪ check_for_selection() and self.master.frames[StudentPage].
    ↪ check_for_selection():
128             self.runButton["state"]="enabled"
129             self.warning.pack_forget()
130 
131         #Otherwise, disable the run button and display the warning
    ↪ message
132         else:
133             self.runButton["state"]="disabled"
134             self.warning.pack()
135 
136     def get_data_for_algo(self,*args,**kwargs):
137         database=DATABASE
138         projects=self.master.frames[ProjectPage].get_selected()
139         students=self.master.frames[StudentPage].get_selected()
140         settings=self.master.frames[SettingsPage].get_settings()
141         fitness=settings["fitness"]
142         iterations=settings["iterations"]
143         popSize=settings["popSize"]
144 
145         self.master.show_frame(LoadingPage,database,students,projects,
    ↪ fitness,iterations,popSize)
146 
147 
148 
149 
150 
151 class ProjectPage(tk.Frame):
152     def __init__(self,parent):
153         #Set up the project page layout
154         tk.Frame.__init__(self,parent)
155 
156         self.header=Header(self,"Projects",HEADER_COLOUR)
157         self.header.pack(fill="x")
158 
159         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
160         self.backButton.pack(side="right")
```

```
161
162         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_projects)
163         self.importButton.pack()
164
165         self.addButton=ttk.Button(self,text="New project",command=lambda:
    ↪ self.create_entry_window("New"))
166         self.addButton.pack()
167
168         #self.separator=ttk.Separator(self,orient="horizontal")
169         #self.separator.pack(fill="x",expand=True)
170
171         self.boxesFrame=ScrollableFrame(self)
172         self.boxesFrame.canvas.pack(fill="both",expand=True)
173         self.update()
174
175         #Get and show project data from the database
176         self.projectBoxes={}
177         self.fetch_from_db(DATABASE)
178
179
180     def fetch_from_db(self,path):
181         #Fetch all project data from a database
182         query="SELECT * from Projects LEFT JOIN ProjectSkills using (
    ↪ ProjectID)"
183         with sql.connect(path) as conn:
184             cur = conn.cursor()
185             cur.execute(query)
186             projects=cur.fetchall()
187
188             for project in projects:
189                 ID=project[0]
190                 #When a project has multiple skills, the ID is repeated
    ↪ in the recieved tuple for each skill
191                 if ID in self.projectBoxes:
192                     #The next skill is appended to the existing list of
    ↪ skills for this ID
193                     data=self.projectBoxes[ID].get_data()
194
195                     skills=data["skills"]
196                     skills.append(project[5])
197                     self.projectBoxes[ID].edit_data(skills=skills)
198
199                 #If the project has not been added to the dictionary yet,
    ↪  add it
200                 else:
201                     #Unpack the data, keeping data that should not be of
    ↪ type string spearate
202                     *data,optional,skills=project
203                     #Convert to correct data type
```

```
204                     optional=bool(optional)
205                     skills=[skills]
206                     #Store data and display data on UI
207                     self.projectBoxes[ID]=projectBox(self.boxesFrame,self
    ↪ ,*data,optional,skills)
208                     self.projectBoxes[ID].pack()
209
210
211     #Method override as a work-around for a scrolling bug
212     def update(self):
213         self.master.state('normal')
214         tk.Frame.update(self)
215         self.master.state('zoomed')
216
217     #Iterates through each project, returning true as soon as it finds a
    ↪ selected project. Returns false when no project is selected.
218     def check_for_selection(self):
219         for box in self.projectBoxes:
220             if self.projectBoxes[box].check_selection():
221                 return True
222         return False
223
224     def get_selected(self):
225         sel=set()
226         for box in self.projectBoxes:
227             if self.projectBoxes[box].check_selection():
228                 sel.add(box)
229         return sel
230
231
232     def create_entry_window(self,mode,**kwargs):
233         name=tk.StringVar()
234         maxSize=tk.StringVar()
235         minSize=tk.StringVar()
236         optional=tk.BooleanVar()
237         skills=tk.StringVar()
238         data={"name":name,"maxSize":maxSize,"minSize":minSize,"optional":
    ↪ optional,"skills":skills}
239
240         if mode == "Edit":
241             popupWin=entryPopupWin(self,"Edit Project")
242             name.set(kwargs["name"])
243             maxSize.set(kwargs["maxSize"])
244             minSize.set(kwargs["minSize"])
245             optional.set(kwargs["optional"])
246             skills.set(", ".join(kwargs["skills"]))
247             data["ID"]=kwargs["ID"]
248         else:
249             popupWin=entryPopupWin(self,"New project")
250
```

```
251         popupWin.grab_set()
252         popupWin.add_string_entry("Name:",name)
253 
254         popupWin.add_int_entry("Maximum size allowed for this project's
    ↪ group:",maxSize)
255 
256         popupWin.add_int_entry("Minimum size allowed for this project's
    ↪ group:",minSize)
257 
258         popupWin.add_checkbox("Optional:",optional)
259 
260         popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
261 
262         popupWin.add_save_cancel(popupWin,mode,**data)
263 
264     def validate_entry(self,name,maxSize,minSize,optional,skills):
265         if name != "" and maxSize != "" and minSize != "" and optional !=
    ↪  "" and skills != "":
266             try:
267                 maxSize = int(maxSize)
268                 minSize = int(minSize)
269             except:
270                 return {"valid":False,"reason":"Minimum group size and
    ↪ maximum group size must be integers"}
271             else:
272                 if minSize>0:
273                     if maxSize>=minSize:
274                         return {"valid":True}
275                     else:
276                         return {"valid":False,"reason":"Maximum group
    ↪ size must be greater than or equal to minimum group size."}
277                 else:
278                     return {"valid":False,"reason":"Minimum group size
    ↪ must be greater than 0."}
279         else:
280             return {"valid":False,"reason":"All entries must be filled."}
281 
282     def save_entry(self,window,mode,**kwargs):
283         name=kwargs["name"].get()
284         maxSize=kwargs["maxSize"].get()
285         minSize=kwargs["minSize"].get()
286         optional=kwargs["optional"].get()
287         skills=kwargs["skills"].get()
288 
289         valid=self.validate_entry(name,maxSize,minSize,optional,skills)
290         if valid["valid"]:
291             #Separate skills into a list, remove whitespace, and add to a
    ↪  set to remove duplicates
292             skillList=skills.split(",")
```

```
            skills=set()
            for i in range(len(skillList)):
                skills.add(skillList[i].strip())

            if mode == "Edit":
                self.edit_project(kwargs["ID"],name,maxSize,minSize,
    ↪ optional,skills)
            else:
                self.new_project(name,maxSize,minSize,optional,skills)

            #Destroy the popup window
            window.destroy()

        else:
            messagebox.showerror(title="Error",message=valid["reason"])


    def import_projects(self):


        def importtxt(path):
            file=open(path,mode="r")
            file.close()


        def importcsv(path):
            file=open(path,newline='')
            reader=csv.reader(file)
            isRow1=True
            for row in reader:
                if isRow1 and not row==["name, maxSize, minSize, optional
    ↪ , skills"]:
                    messagebox.showerror(title="Import Error",message="
    ↪ CSV File not in correct format.")
                    break
                isRow1=False

                valid=self.validate_entry(*row)
                if valid["valid"]:
                    #Separate skills into a list, remove whitespace, and
    ↪ add to a set to remove duplicates
                    skillList=row[4].split(",")
                    skills=set()
                    for i in range(len(skillList)):
                        skills.add(skillList[i].strip())

                    self.new_project(row[0],row[1],row[2],row[3],skills)
                else:
                    messagebox.showerror(title="Import Error",message=f"
    ↪ Import failed. {valid["reason"]}")
```

```
338
339
340         def importdb(path):
341             return
342
343
344         #Show an "Open" dialog box and return file directory
345         path = filedialog.askopenfilename()
346         #Validation to ensure the user selected a valid file type
347         if path != None:
348             match path[-4:]:
349                 #case ".txt":
350                     #importtxt(path)
351                 case ".csv":
352                     importcsv(path)
353                 case _:
354                     #if checkForDB(path):
355                         #importdb(path)
356                     #else:
357                     messagebox.showerror(title="Import Error", message="
    ↪ File type unrecognised!")
358
359         else:
360             messagebox.showerror(title="Import Error", message="No file
    ↪ selected!")
361
362
363
364     #Add the inputted data to the Project and Project Skills tables in
    ↪ the database
365     def new_project(self,name,maxSize,minSize,optional,skills):
366         mainquery="INSERT INTO Projects (Name, MaxSize, MinSize, Optional
    ↪ ) VALUES (?,?,?,?)"
367         skillsquery="INSERT INTO ProjectSkills (ProjectID, Skill) VALUES
    ↪ (?,?)"
368         with sql.connect(DATABASE) as conn:
369             cur = conn.cursor()
370
371             cur.execute(mainquery,(name,maxSize,minSize,optional))
372
373             #Fetch the autogenerated ID for this student
374             ID=cur.lastrowid
375
376             for skill in skills:
377                 cur.execute(skillsquery,(ID, skill))
378
379             conn.commit()
380
381         #Display new project on the GUI
```

```
382         self.projectBoxes[ID]=projectBox(self.boxesFrame,self,ID,name,
    ↪ maxSize,minSize,optional,skills)
383         self.projectBoxes[ID].pack()
384 
385         #Work-around for a scrollbar bug
386         self.update()
387 
388     def edit_project(self,ID,name,maxSize,minSize,optional,skills):
389         edited=False
390         box=self.projectBoxes[ID]
391         oldData=box.get_data()
392 
393         #Only updates if the new data is different from the old data
394         with sql.connect(DATABASE) as conn:
395             cur = conn.cursor()
396 
397             if name != oldData["name"]:
398                 edited=True
399 
400                 query = "UPDATE Projects SET Name = ? WHERE ProjectID = ?
    ↪ "
401                 cur.execute(query,(name,ID))
402 
403                 box.edit_data(name=name)
404 
405             if maxSize != oldData["maxSize"]:
406                 edited=True
407 
408                 query="UPDATE Projects SET MaxSize = ? WHERE ProjectID =
    ↪ ?"
409                 cur.execute(query,(maxSize,ID))
410 
411                 box.edit_data(maxSize=maxSize)
412 
413             if minSize != oldData["minSize"]:
414                 edited=True
415 
416                 query="UPDATE Projects SET MinSize = ? WHERE ProjectID =
    ↪ ?"
417                 cur.execute(query,(minSize,ID))
418 
419                 box.edit_data(minSize=minSize)
420 
421             if optional != oldData["optional"]:
422                 edited=True
423 
424                 query="UPDATE Projects SET Optional = ? WHERE ProjectID =
    ↪  ?"
425                 cur.execute(query,(optional,ID))
426 
```

```
                box.edit_data(optional=optional)

            if skills != oldData["skills"]:
                edited=True

                cur.execute("DELETE FROM ProjectSkills WHERE ProjectID =
    ↪ ?",(ID,))
                query = "INSERT INTO ProjectSkills (ProjectID, Skill)
    ↪ VALUES (?,?)"
                for skill in skills:
                    cur.execute(query,(ID,skill))

                box.edit_data(skills=skills)

            if edited:
                self.update()
                conn.commit()

    def editBox(self,boxID):
        box=self.projectBoxes[boxID]
        data=box.get_data()
        self.create_entry_window("Edit",ID=data["ID"],name=data["name"],
    ↪ maxSize=data["maxSize"],minSize=data["minSize"], optional=data["
    ↪ optional"], skills=data["skills"])

    def deleteBox(self,boxID):
        #Delete from database
        query1 = "DELETE FROM Projects WHERE ProjectID=?"
        query2 = "DELETE FROM ProjectSkills WHERE ProjectID=?"
        with sql.connect(DATABASE) as conn:
            cur = conn.cursor()
            cur.execute(query1,(boxID,))
            cur.execute(query2,(boxID,))
            conn.commit()
        #Delete from UI
        self.projectBoxes[boxID].pack_forget()
        #Delete from internal store
        del self.projectBoxes[boxID]

        self.update()



class StudentPage(tk.Frame):
    def __init__(self,parent):
        #Set up student page layout
        tk.Frame.__init__(self, parent)

        self.header=Header(self,"Students",HEADER_COLOUR)
        self.header.pack(fill="x")
```

```
473
474         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
475         self.backButton.pack(side="right")
476 
477         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_students)
478         self.importButton.pack()
479 
480         self.addButton=ttk.Button(self,text="New student",command=lambda:
    ↪ self.create_entry_window("New"))
481         self.addButton.pack()
482 
483         self.boxesFrame=ScrollableFrame(self)
484         self.boxesFrame.canvas.pack(fill="both",expand=True)
485         self.update()
486 
487         self.studentBoxes={}
488 
489         #Fetch all student data from the database
490         studentquery="SELECT * from Students LEFT JOIN StudentSkills
    ↪ using (StudentID)"
491         with sql.connect(DATABASE) as conn:
492             cur = conn.cursor()
493             cur.execute(studentquery)
494             students=cur.fetchall()
495 
496             for student in students:
497                 ID=student[0]
498                 #When a student has multiple skills, the ID is repeated
    ↪ in the recieved tuple for each skill
499                 if ID in self.studentBoxes:
500                     #The next skill is appended to the existing list of
    ↪ skills for this ID
501                     data=self.studentBoxes[ID].get_data()
502                     skills=data["skills"]
503                     skills.append(student[3])
504                     self.studentBoxes[ID].edit_data(skills=skills)
505                 #If the student has not been added to the dictionary yet,
    ↪  add it
506                 else:
507                     #Unpack the data, keeping data that should not be of
    ↪ type string spearate
508                     *data,skills=student
509                     #Convert to correct data type
510                     skills=[skills]
511                     #Store data and display on UI
512                     self.studentBoxes[ID]=studentBox(self.boxesFrame,self
    ↪ ,*data,skills)
513                     self.studentBoxes[ID].pack()
```

```
514
515     #Method override as a work-around for a scrolling bug
516     def update(self):
517         self.master.state('normal')
518         tk.Frame.update(self)
519         self.master.state('zoomed')
520
521     def check_for_selection(self):
522         for box in self.studentBoxes:
523             if self.studentBoxes[box].check_selection():
524                 return True
525         return False
526
527     def get_selected(self):
528         sel=set()
529         for box in self.studentBoxes:
530             if self.studentBoxes[box].check_selection():
531                 sel.add(box)
532         return sel
533
534     def validate_entry(self,name,gender,skills):
535         if name != "" and gender != "" and skills != "":
536             return {"valid":True}
537         else:
538             return {"valid":False,"reason":"All entries must be filled."}
539
540
541     def import_students(self):
542
543         def importcsv(path):
544             file=open(path,newline='')
545             reader=csv.reader(file)
546             isRow1=True
547             for row in reader:
548                 if isRow1 and not row==["name, gender, skills"]:
549                     messagebox.showerror(title="Import Error",message="
    ↪ CSV File not in correct format.")
550                     break
551                 isRow1=False
552
553                 valid=self.validate_entry(*row)
554                 if valid["valid"]:
555                     #Separate skills into a list, remove whitespace, and
    ↪ add to a set to remove duplicates
556                     skillList=row[2].split(",")
557                     skills=set()
558                     for i in range(len(skillList)):
559                         skills.add(skillList[i].strip())
560
561                     self.new_project(row[0],row[1],skills)
```

```
562                 else:
563                     messagebox.showerror(title="Import Error",message=f"
    ↪ Import failed. {valid["reason"]}")
564 
565 
566         #Show an "Open" dialog box and return an open file object in read
    ↪  mode
567         path = filedialog.askopenfilename()
568         #Validation to ensure the user selected a valid file type
569         if path != None:
570             match path[-4:]:
571                 #case ".txt":
572                     #importtxt(path)
573                 case ".csv":
574                     importcsv(path)
575                 case _:
576                     #if checkForDB(path):
577                         #importdb(path)
578                     #else:
579                     messagebox.showerror(title="Import Error", message="
    ↪ File type unrecognised!")
580 
581         else:
582             messagebox.showerror(title="Import Error", message="No file
    ↪ selected!")
583 
584     def create_entry_window(self,mode,**kwargs):
585         name=tk.StringVar()
586         gender=tk.StringVar()
587         skills=tk.StringVar()
588         data={"name":name,"gender":gender,"skills":skills}
589 
590         if mode == "Edit":
591             popupWin=entryPopupWin(self, "Edit student")
592             name.set(kwargs["name"])
593             gender.set(kwargs["gender"])
594             skills.set(", ".join(kwargs["skills"]))
595             data["ID"]=kwargs["ID"]
596         else:
597             popupWin=entryPopupWin(self,"New student")
598 
599         popupWin.grab_set()
600 
601         popupWin.add_string_entry("Name:", name)
602 
603         popupWin.add_string_entry("Gender:",gender)
604 
605         popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
606 
```

```
607         popupWin.add_save_cancel(popupWin,mode,**data)
608 
609     def save_entry(self,window,mode,**kwargs):
610         name=kwargs["name"].get()
611         gender=kwargs["gender"].get()
612         skills=kwargs["skills"].get()
613 
614         #Validation to ensure all entries meet standards
615         valid=self.validate_entry(name,gender,skills)
616         if valid["valid"]:
617             #Separate skills into a list, remove whitespace, add to set
    ↪ to remove duplicates
618             skillList=skills.split(",")
619             skills=set()
620             for i in range(len(skillList)):
621                 skills.add(skillList[i].strip())
622 
623             if mode == "Edit":
624                 self.edit_student(kwargs["ID"],name,gender,skills)
625             else:
626                 self.new_student(name,gender,skills)
627 
628             #Destroy the popup window
629             window.destroy()
630         else:
631             messagebox.showerror(title="Error",message=valid["reason"])
632 
633 
634     #Add the inputted data to the Student and StudentSkills tables in the
    ↪  database
635     def new_student(self,name,gender,skills):
636         mainquery="INSERT INTO Students (Name, Gender) VALUES (?,?)"
637         skillsquery="INSERT INTO StudentSkills (StudentID, Skill) VALUES
    ↪ (?,?)"
638         with sql.connect(DATABASE) as conn:
639             cur = conn.cursor()
640 
641             cur.execute(mainquery,(name,gender))
642 
643             #Fetch the autogenerated ID for this student
644             ID=cur.lastrowid
645 
646             for skill in skills:
647                 cur.execute(skillsquery,(ID, skill))
648 
649             conn.commit()
650 
651         #Display new student on the GUI
652         self.studentBoxes[ID]=studentBox(self.boxesFrame,self,ID,name,
    ↪ gender,skills)
```

```
653             self.studentBoxes[ID].pack()
654 
655             #Scroll bar bug work-around
656             self.update()
657 
658 
659         def edit_student(self,ID,name,gender,skills):
660             edited=False
661 
662             box=self.studentBoxes[ID]
663             oldData=box.get_data()
664 
665             #Only updates if the new data is different from the old data
666             with sql.connect(DATABASE) as conn:
667                 cur = conn.cursor()
668 
669                 if name != oldData["name"]:
670                     edited=True
671 
672                     query = "UPDATE Students SET Name = ? WHERE StudentID = ?
    ↪ "
673                     cur.execute(query,(name,ID))
674 
675                     box.edit_data(name=name)
676 
677                 if gender != oldData["gender"]:
678                     edited=True
679 
680                     query="UPDATE Students SET Gender = ? WHERE StudentID = ?
    ↪ "
681                     cur.execute(query,(gender,ID))
682 
683                     box.edit_data(gender=gender)
684 
685                 if skills != oldData["skills"]:
686                     edited=True
687 
688                     cur.execute("DELETE FROM StudentSkills WHERE StudentID =
    ↪ ?",(ID,))
689                     query = "INSERT INTO StudentSkills (StudentID, Skill)
    ↪ VALUES (?,?)"
690                     for skill in skills:
691                         cur.execute(query,(ID,skill))
692 
693                     box.edit_data(skills=skills)
694 
695                 if edited:
696                     self.update()
697                     conn.commit()
698 
```

```
699     def editBox(self,boxID):
700         box=self.studentBoxes[boxID]
701         data=box.get_data()
702         self.create_entry_window("Edit", ID=data["ID"],name=data["name"],
    ↪ gender=data["gender"],skills=data["skills"])
703 
704     def deleteBox(self,boxID):
705         #Delete from database
706         query1 = "DELETE FROM Students WHERE StudentID=?"
707         query2 = "DELETE FROM StudentSkills WHERE StudentID=?"
708         with sql.connect(DATABASE) as conn:
709             cur = conn.cursor()
710             cur.execute(query1,(boxID,))
711             cur.execute(query2,(boxID,))
712             conn.commit()
713         #Delete from UI
714         self.studentBoxes[boxID].pack_forget()
715         #Delete from internal store
716         del self.studentBoxes[boxID]
717 
718         self.update()
719 
720 
721 
722 class LoadingPage(tk.Frame):
723     def __init__(self,parent):
724         #Set up loading page layout
725         tk.Frame.__init__(self, parent)
726 
727         self.label = ttk.Label(self, text="Grouping in progress...")
728         self.label.pack()
729 
730         self.cancelButton=ttk.Button(self,text="Cancel",command=lambda:
    ↪ parent.show_frame(HomePage))
731         self.cancelButton.pack()
732 
733     #Method override that runs once everytime the page is opened
734     #Runs the genetic algorithm
735     def tkraise(self, database, studentIDs, projectIDs, tgtFit, maxIters,
    ↪  popSize,*args,**kwargs):
736         tk.Frame.tkraise(self)
737         results=ga.run(database,studentIDs,projectIDs,tgtFit,maxIters,
    ↪ popSize)
738         if results["status"]==-1:
739             messagebox.showerror("Error",results["error"])
740         elif results["status"]==0:
741             results=results["results"]
742             self.master.show_frame(ResultsPage,results)
743         else:
744             messagebox.showerror("Error","An unknown error occured")
```

```
745
746
747 class ResultsPage(tk.Frame):
748     def __init__(self,parent):
749         #Set up results page layout
750         tk.Frame.__init__(self, parent)
751
752         self.results=None
753         self.resultBoxes={}
754
755         self.header=Header(self,"Results",HEADER_COLOUR)
756         self.header.pack(fill="x")
757
758         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
759         self.homeButton.pack(side="right")
760
761         self.registerButton=ttk.Button(self.header,text="Switch to
    ↪ register layout",style="Header.TButton",command=lambda:parent.
    ↪ show_frame(RegisterPage,self.results))
762         self.registerButton.pack(side="right")
763
764         #Allows scrolling through the boxes
765         self.boxesFrame=ScrollableFrame(self)
766         self.boxesFrame.canvas.pack(fill="both",expand=True)
767         self.update()
768
769     def tkraise(self,results,*args,**kwargs):
770         tk.Frame.tkraise(self)
771
772         while len(self.resultBoxes)>0:
773             ID=next(iter(self.resultBoxes))
774             self.resultBoxes[ID].pack_forget()
775             del self.resultBoxes[ID]
776
777         self.results=results
778         #Stores the boxes shown in the scrollable frame
779         self.resultBoxes={}
780
781         #Create the boxes and display project data in them
782         for projectID in results:
783             result=results[projectID]
784             self.resultBoxes[projectID]=resultBox(self.boxesFrame,self,
    ↪ projectID,result["name"],result["students"],result["size"],result
    ↪ ["genders"],result["skills"])
785             self.resultBoxes[projectID].pack()
786
787         self.update()
788
789     #Method override as a work-around for a scrolling bug
```

```
790     def update(self):
791         self.master.state('normal')
792         tk.Frame.update(self)
793         self.master.state('zoomed')
794 
795 
796 class RegisterPage(tk.Frame):
797     def __init__(self,parent):
798         #Set up register page layout
799         tk.Frame.__init__(self, parent)
800         self.columnconfigure(0,weight=1)
801         self.columnconfigure(1,weight=1)
802         self.columnconfigure(2,weight=2)
803 
804         self.results=None
805         self.boxesFrame=None
806         self.registerBoxes={}
807 
808         self.header=Header(self,"Register",HEADER_COLOUR)
809         self.header.pack(fill="x")
810 
811         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
812         self.homeButton.pack(side="right")
813 
814         self.resultsButton=ttk.Button(self.header,text="Switch to results
    ↪  layout",style="Header.TButton",command=lambda:parent.show_frame(
    ↪ ResultsPage,self.results))
815         self.resultsButton.pack(side="right")
816 
817     def tkraise(self,results,*args,**kwargs):
818         tk.Frame.tkraise(self)
819 
820         #Prevent old data from preventing new data from appearing on the
    ↪ GUI
821         if self.boxesFrame!=None:
822             self.boxesFrame.pack_forget()
823 
824         while len(self.registerBoxes)>0:
825             ID=next(iter(self.registerBoxes))
826             self.registerBoxes[ID].place_forget()
827             del self.registerBoxes[ID]
828 
829         self.results=results
830 
831         #This frame contains the groups
832         self.boxesFrame=tk.Frame(self)
833         self.boxesFrame.pack(fill="both",expand=True)
834         #Stores the boxes shown in the frame
835         self.registerBoxes={}
```

```
836
837         #Find grid size
838         keys=list(results)
839         root=len(keys)**(0.5)
840         if root-int(root)>=0.5:
841             y=(int(root)+1)
842         else:
843             y=(int(root))
844         x=(int(root)+1)
845 
846         #Iterate through data in alphabetical order by project name
847         i=0
848         for projectID, project in sorted(results.items(), key=lambda item
    ↪ : item[1]["name"]):
849             #Create boxes with student names in alphabetical order
850             self.registerBoxes[projectID]=registerBox(self.boxesFrame,
    ↪ self,projectID,project["name"],sorted(project["students"]))
851             #Show boxes on GUI in a grid pattern
852             self.registerBoxes[projectID].place(relx=((i%x)/x)*(1.003),
    ↪ rely=((i//y)/y)*(1.003),relwidth=(1/x)*(0.997),relheight=(1/y)*(0
    ↪ .997))
853             i+=1
854 
855 
856 class SettingsPage(tk.Frame):
857     def __init__(self,parent,fitness,iterations,popSize):
858         #Set up settings page layout
859         tk.Frame.__init__(self,parent)
860         self.fitness=fitness
861         self.iterations=iterations
862         self.popSize=popSize
863 
864         self.header=Header(self,"Settings",HEADER_COLOUR)
865         self.header.pack(fill="x")
866 
867         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=self.go_back)
868         self.backButton.pack(side="right")
869 
870         self.fitnessVar=tk.DoubleVar(value=self.fitness)
871         self.iterVar=tk.IntVar(value=self.iterations)
872         self.popSizeVar=tk.IntVar(value=self.popSize)
873 
874         ttk.Button(self,text="Export database", command=self.export_db).
    ↪ pack()
875 
876         #Only allows user to type a decimal number between 0 and 1, no
    ↪ other characters
877         def fitnessCallback(P):
```

```
878             if re.match("^0(\\.\\d*)?$",P) or re.match("^1(\\.0*)?$",P)
    ↪ or P == "":
879                 return True
880             return False
881 
882         #Only allows the user to enter positive integers
883         def posIntCallback(P):
884             return str.isdigit(P) or P==""
885 
886         fitnessValidate = (self.register(fitnessCallback), '%P')
887         posIntValidate = (self.register(posIntCallback), '%P')
888 
889         ttk.Label(self, text="Target fitness for genetic algorithm (Must
    ↪ be between 0-1).").pack()
890         ttk.Label(self,text="Higher target fitness means better groups,
    ↪ with 1 being perfect groupings. May not be reached if maximum
    ↪ iterations is reached").pack()
891         ttk.Spinbox(self, textvariable=self.fitnessVar,validate="all",
    ↪ validatecommand=fitnessValidate,from_=0,increment=0.05,to=1).pack
    ↪ ()
892 
893         ttk.Label(self,text="Maximum iterations the genetic algorithm
    ↪ could complete (may be less if target fitness is reached).").pack
    ↪ ()
894         ttk.Spinbox(self,textvariable=self.iterVar,validate="all",
    ↪ validatecommand=posIntValidate,from_=1,increment=5,to=100).pack()
895 
896         ttk.Label(self,text="Size of populations for genetic algorithm.
    ↪ Greater sizes leads to longer runtime, but better results.").pack
    ↪ ()
897         ttk.Spinbox(self,textvariable=self.popSizeVar,validate="all",
    ↪ validatecommand=posIntValidate,from_=1,increment=10,to=200).pack
    ↪ ()
898 
899         ttk.Button(self, text="Save", command=self.save_settings).pack()
900 
901     #Method override that runs once everytime this page is opened
902     def tkraise(self,*args,**kwargs):
903         self.fitnessVar.set(self.fitness)
904         self.iterVar.set(self.iterations)
905         self.popSizeVar.set(self.popSize)
906         tk.Frame.tkraise(self)
907 
908     def get_settings(self):
909         return {"fitness":self.fitness,"iterations":self.iterations,"
    ↪ popSize":self.popSize}
910 
911     #Command run by the back button
912     def go_back(self):
```

```
913             choice=messagebox.askyesno("Warning","Going back will not save
    ↪ your changes. Do you still wish to proceed?",icon="warning")
914             if choice:
915                 self.master.show_frame(HomePage)
916 
917         #Validate data entry
918         def validate(self):
919             try:
920                 popSize=int(self.popSizeVar.get())
921                 iterVar=int(self.iterVar.get())
922             except:
923                 messagebox.showerror(title="Error",message="")
924                 return False
925             else:
926                 if popSize>=4:
927                     if iterVar>0:
928                         return True
929                     else:
930                         messagebox.showerror(title="Error",message="Max
    ↪ iterations must be greater than 0")
931                         return False
932                 else:
933                     messagebox.showerror(title="Error",message="Population
    ↪ size must be greater than or equal to 4")
934                     return False
935             return False
936 
937 
938         #Save settings input if valid
939         def save_settings(self):
940             valid=self.validate()
941             if valid:
942                 self.fitness=float(self.fitnessVar.get())
943                 self.iterations=int(self.iterVar.get())
944                 self.popSize=int(self.popSizeVar.get())
945                 self.master.show_frame(HomePage)
946 
947         #The ui for selecting details before selecting directory
948         #Would be used for selecting file type to save as
949         def export_db(self):
950             self.write_csv()
951 
952         #Save database to a csv file
953         def write_csv(self):
954             filetype=[("CSV file",".csv")]
955             file = filedialog.asksaveasfile(filetypes=filetype,
    ↪ defaultextension=filetype)
956             if file is not None:
957                 try:
958                     with sql.connect(DATABASE) as conn:
```

```
                    cur = conn.cursor()
                    #Fetch names of every table
                    cur.execute("SELECT name FROM sqlite_schema WHERE
    ↪ type ='table' AND name NOT LIKE 'sqlite_%';")
                    tables=cur.fetchall()
                    #Fetch all the data from each table
                    for table in tables:
                        data=cur.execute("SELECT * FROM "+str(table[0]))

                        #Write to the csv
                        writer = csv.writer(file)
                        writer.writerow("")
                        writer.writerow(table)
                        #Get attribute names
                        cols=[]
                        for col in data.description:
                            cols.append(col[0])
                        writer.writerow(cols)
                        writer.writerows(data)
            except:
                messagebox.showerror(title="Error", message="An error
    ↪ occured.")
        else:
            messagebox.showerror(title="Error",message="No file selected.
    ↪ ")
        file.close()

#=====WIDGETS=====

class Header(tk.Frame):
    def __init__(self,parent,text,colour):
        tk.Frame.__init__(self,parent)
        self.config(bg=colour)
        self.label = tk.Label(self, text=text, bg=colour,font=LARGE_FONT)
        self.label.pack(side="left")

class ScrollableFrame(tk.Frame):
    def __init__(self,parent):
        #Canvas is the only tkinter widget which supports both scrollbars
    ↪  and embedded widgets
        self.canvas=tk.Canvas(parent,background="white")
        tk.Frame.__init__(self,self.canvas)

        self.scrollbar=tk.Scrollbar(self.canvas,orient="vertical",command
    ↪ =self.canvas.yview)

        self.canvas.configure(yscrollcommand=self.scrollbar.set)

        self.scrollbar.pack(side="right",fill="y")
        self.scrollbar.update()
```

```
1004
1005            #Assign the scrollbar and scrolling function to the canvas
1006            self.canvas.create_window((0,0),window=self,anchor='nw',width=
        ↪ self.winfo_screenwidth()-self.scrollbar.winfo_width())
1007            self.canvas.bind("<Configure>",self.scroll)
1008
1009
1010        def scroll(self,event):
1011            self.canvas.configure(scrollregion=self.canvas.bbox("all"),width=
        ↪ self.winfo_width(),height=self.winfo_height())
1012
1013    #Has checkbox, name, edit button, delete button.
1014    class selectionBox(tk.Frame):
1015        def __init__(self,parent,page,ID,name):
1016            tk.Frame.__init__(self,parent,relief="solid",bd=2)
1017
1018            self.ID=ID
1019            self.name=name
1020            self.page=page
1021
1022            self.selected=tk.BooleanVar(value=False)
1023            self.checkbox=ttk.Checkbutton(self,variable=self.selected,onvalue
        ↪ =True,offvalue=False)
1024            self.checkbox.pack()
1025
1026            self.nameLabel=ttk.Label(self,text=name)
1027            self.nameLabel.pack()
1028
1029            self.editButton=ttk.Button(self,text="Edit",command=self.edit)
1030            self.editButton.pack()
1031
1032            self.deleteButton=ttk.Button(self,text="Delete",command=self.
        ↪ delete)
1033            self.deleteButton.pack()
1034
1035        #Return whether this item has been selected by the user
1036        def check_selection(self):
1037            return self.selected.get()
1038
1039        #Opens the edit window, which is handled by the page the box is
        ↪ located within
1040        def edit(self):
1041            self.page.editBox(self.ID)
1042
1043        #Ask for confirmation before deletion
1044        #Deletion is handled by the page the box is located on
1045        def delete(self):
1046            delChoice=messagebox.askyesno(title="Delete entry", message=f"Are
        ↪  you sure you want to delete {self.name}?")
1047            if delChoice:
```

```
1048            #Ensure deleted data can never get caught up in the algorithm
1049            self.selected.set(False)
1050            self.page.deleteBox(self.ID)
1051
1052    #Ensure all boxes follow the same design
1053    def pack(self):
1054        tk.Frame.pack(self,fill="x",padx=10,pady=5)
1055
1056    #-----Abstract methods-----
1057    def get_data(self):
1058        return
1059
1060    def edit_data(self):
1061        return
1062
1063
1064 class projectBox(selectionBox):
1065    def __init__(self,parent,page,ID,name,maxSize,minSize,optional,skills
       ↪ ):
1066        selectionBox.__init__(self,parent,page,ID,name)
1067        self.maxSize=maxSize
1068        self.minSize=minSize
1069        self.optional=optional
1070        self.skills=skills
1071
1072        self.maxSizeLabel=ttk.Label(self,text=f"Max group size: {maxSize}
       ↪ ")
1073        self.maxSizeLabel.pack()
1074
1075        self.minSizeLabel=ttk.Label(self,text=f"Min group size: {minSize}
       ↪ ")
1076        self.minSizeLabel.pack()
1077
1078        self.optionalLabel=ttk.Label(self,text=f"Optional: {optional}")
1079        self.optionalLabel.pack()
1080
1081        self.skillsLabel=ttk.Label(self,text=f"Skills: {", ".join(skills)
       ↪ }")
1082        self.skillsLabel.pack()
1083
1084    #Return all the data about the project
1085    def get_data(self):
1086        return {"ID":self.ID,"name":self.name,"maxSize":self.maxSize,"
       ↪ minSize":self.minSize,"optional":self.optional,"skills":self.
       ↪ skills}
1087
1088    #After the user has edited some project data, this method updates its
       ↪  data stores and changes the data shown on the GUI
1089    def edit_data(self,**kwargs):
1090        if "name" in kwargs:
```

```
1091                self.name=kwargs["name"]
1092                self.nameLabel.config(text=self.name)
1093            if "maxSize" in kwargs:
1094                self.maxSize=kwargs["maxSize"]
1095                self.maxSizeLabel.config(text=f"Max group size: {self.maxSize
    ↪ }")
1096            if "minSize" in kwargs:
1097                self.minSize=kwargs["minSize"]
1098                self.minSizeLabel.config(text=f"Min group size: {self.minSize
    ↪ }")
1099            if "optional" in kwargs:
1100                self.optional=kwargs["optional"]
1101                self.optionalLabel.config(text=f"Optional: {self.optional}")
1102            if "skills" in kwargs:
1103                self.skills=kwargs["skills"]
1104                self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
    ↪ )}")
1105
1106    class studentBox(selectionBox):
1107        def __init__(self, parent, page, ID, name, gender, skills):
1108            selectionBox.__init__(self,parent,page,ID,name)
1109            self.gender=gender
1110            self.skills=skills
1111
1112            self.genderLabel = ttk.Label(self,text=f"Gender: {gender}")
1113            self.genderLabel.pack()
1114
1115            self.skillsLabel = ttk.Label(self, text= f"Skills: {", ".join(
    ↪ skills)}")
1116            self.skillsLabel.pack()
1117
1118        #Returns all data about the student
1119        def get_data(self):
1120            return {"ID":self.ID,"name":self.name,"gender":self.gender,"
    ↪ skills":self.skills}
1121
1122        #After the user has edited some student data, this method updates its
    ↪  data stores and changes the data shown on the GUI
1123        def edit_data(self,**kwargs):
1124            if "name" in kwargs:
1125                self.name=kwargs["name"]
1126                self.nameLabel.config(text=self.name)
1127            if "gender" in kwargs:
1128                self.gender=kwargs["gender"]
1129                self.genderLabel.config(text=f"Gender: {self.gender}")
1130            if "skills" in kwargs:
1131                self.skills=kwargs["skills"]
1132                self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
    ↪ )}")
1133
```

```
#Customisable pop-up widget where different types of entries and buttons can be added dynamically
class entryPopupWin(tk.Toplevel):
    def __init__(self,parent,title):
        tk.Toplevel.__init__(self,parent)
        self.title(title)

    def add_string_entry(self,text,variable):
        ttk.Label(self, text=text).pack()
        ttk.Entry(self, textvariable=variable).pack()

    def add_int_entry(self,text,variable,minimum=0,maximum=100,increment=1):

        #Prevents user from typing non numbers into number-only entry boxes
        def intCallback(P):
            return str.isdigit(P) or P == ""

        intValidate = (self.register(intCallback), '%P')

        ttk.Label(self, text=text).pack()
        ttk.Spinbox(self, textvariable=variable,validate="all", validatecommand=intValidate,from_=minimum,increment=increment,to=maximum).pack()

    def add_checkbox(self,text,variable):
        ttk.Label(self, text=text).pack()
        ttk.Checkbutton(self, variable=variable, onvalue=True, offvalue=False).pack()

    def add_save_cancel(self,window,mode,**kwargs):
        ttk.Button(self, text="Save", command=lambda:self.master.save_entry(window,mode,**kwargs)).pack()
        ttk.Button(self, text="Cancel", command=self.destroy).pack()


class resultBox(tk.Frame):
    #Used for ensuring adjacent boxes have different colours
    count=0
    def __init__(self,parent,page,ID,name,students:list,size,genders:dict,skills:dict):
        tk.Frame.__init__(self,parent,relief="solid",bd=2)

        #Used for ensuring adjacent boxes have different colours
        resultBox.count+=1
        self.colour=PASTEL_COLOURS[(resultBox.count-1)%6]
        self.config(bg=self.colour)


```

```
1176            self.page=page
1177            self.ID=ID
1178
1179            ttk.Label(self,text=name,background=self.colour).pack(anchor="w")
1180            ttk.Separator(self).pack(fill="x")
1181
1182            ttk.Label(self,text=f"Students: {", ".join(students)}",background
    ↪ =self.colour).pack(anchor="w")
1183            ttk.Label(self,text=f"Group size: {size}",background=self.colour)
    ↪ .pack(anchor="w")
1184
1185            ttk.Label(self,text="Genders:",background=self.colour).pack(
    ↪ anchor="w")
1186            for gender in genders:
1187                ttk.Label(self,text=f"  > {gender}: {genders[gender]}",
    ↪ background=self.colour).pack(anchor="w")
1188
1189            ttk.Label(self,text="Skills:",background=self.colour).pack(anchor
    ↪ ="w")
1190            for skill in skills:
1191                ttk.Label(self,text=f"   > {skill}: {skills[skill]}",
    ↪ background=self.colour).pack(anchor="w")
1192
1193        #Ensure this widget always follows the same design
1194        def pack(self):
1195            tk.Frame.pack(self,fill="x",padx=10,pady=5)
1196
1197    class registerBox(tk.Frame):
1198        #Used for ensuring adjacent boxes have different colours
1199        count=0
1200        def __init__(self,parent,page,ID,name,students:list):
1201            tk.Frame.__init__(self,parent,relief="solid",bd=2)
1202
1203            #Used for ensuring adjacent boxes have different colours
1204            registerBox.count+=1
1205            self.colour=PASTEL_COLOURS[(registerBox.count-1)%6]
1206            self.config(bg=self.colour)
1207
1208            ttk.Label(self,text=name,background=self.colour).pack(anchor="w")
1209            ttk.Separator(self).pack(fill="x")
1210
1211            for student in students:
1212                ttk.Label(self,text=student,background=self.colour).pack(
    ↪ anchor="w")
1213
1214
1215
1216
1217
1218
```

```
1219
1220
1221 #=====SUBROUTINES=====
1222 def checkForDB(path):
1223     try:
1224         #Database files could have any extension, but they all start with
     ↪  the same first 15 characters.
1225         file=open(path,"rb")
1226         line=file.read(15)
1227         file.close()
1228         #Convert binary to string
1229         line="".join(chr(x) for x in line)
1230         if line == "SQLite format 3":
1231             return True
1232         return False
1233     except:
1234         return False
1235
1236
1237 #=====MAIN=====
1238
1239 app=Window()
1240 app.mainloop()
```

3.2 geneticAlgo.py

```
1  import sqlite3 as sql
2  import random
3
4  #=====CLASSES=====
5  class Project:
6      def __init__(self,ID,name,skills,maxSize,minSize,optional):
7          self.ID=ID
8          self.name=name
9          self.skills=skills
10         self.maxSize=maxSize
11         self.minSize=minSize
12         self.optional=optional
13
14         self.group=[]
15
16     #Returns count of genders and skills as dicts, and list of students
17     def get_data(self):
18         genders={}
19         skills={}
20         students=[]
21         for student in self.group:
```

```
            if student.gender in genders:
                genders[student.gender]+=1
            else:
                genders[student.gender]=1

            for skill in student.skills:
                if skill in skills:
                    skills[skill]+=1
                else:
                    skills[skill]=1

            students.append(student.__str__())
        return genders,skills,students

    #Returns fitness value as a float between 0-1
    def getFitness(self):
        fitness=0.0
        stuCount = 0 #Records number of students with at least one relevant skill
        skillsMet=set() #Records the skills the students have that the project demands
        genderCritMet=False #Records whether or not there are multiple genders in the group
        for student in self.group:
            commonSkills=student.skills&self.skills
            #Each student should have one project that is demanded by the project
            if len(commonSkills)>=1:
                stuCount+=1

            #The demands of the project should be met by the students
            skillsMet=skillsMet|commonSkills

            #There should be at least two different genders
            if not genderCritMet and student.gender != self.group[0].gender:
                genderCritMet=True
                fitness+=0.15

        #Prevents a divide by 0 error when the group is empty
        try:
            fitness+=(stuCount/len(self.group))*0.6
        except:
            pass
        finally:
            #Prevents a divide by 0 error if the project has no skill requirements
            try:
                fitness+=(len(skillsMet)/len(self.skills))*0.25
            except:
```

```
                #If the project has no skill requirements, every group has met every skill requirement so earns the max weight
                fitness+=0.25
        return fitness

    #Adds a student to the project's group
    def addStudent(self,students):
        if type(students)==list:
            self.group+=students

        elif type(students)==Student:
            self.group.append(students)

        else:
            raise TypeError(f"Expected data with type list or Student, got data of type {type(students)}")

    #For testing
    def __str__(self):
        string = self.name +", "
        for student in self.group:
            string += student.__str__() +", "
        return string

    #Allows for the same project to appear in different individuals
    def deepcopy(self):
        return Project(self.ID,self.name,self.skills,self.maxSize,self.minSize,self.optional)


class Student:
    def __init__(self,ID,name,skills,gender):
        self.ID=ID
        self.name=name
        self.skills=skills
        self.gender=gender
    def __str__(self):
        return self.name


class Individual:
    def __init__(self):
        self.projects=dict()
        self.fitness=0.0

    #Average the fitness values of each project together to return a float between 0-1
    def calcFitness(self):
        #A fitness of -1 suggests this individual is broken (e.g. does not have every student) and should not be considered by the
```

```
    ↪ algorithm
        if self.fitness==-1:
            return -1

        total=0.000
        for project in self.projects.values():
            total+=project.getFitness()
        try:
            self.fitness = round(total/len(self.projects),3)
        except:
            self.fitness = 0.000
        return self.fitness

    #Adds a project to the individual's list of projects
    def addProject(self,projects):
        if type(projects)==list:
            for project in projects:
                self.projects[project.ID]=project

        elif type(projects)==Project:
            self.projects[projects.ID]=projects

        else:
            raise TypeError(f"Expected data with type list or Project,
    ↪ got data of type {type(projects)}")

    def crossover(self,other,students):
        students=set(students)
        individual=Individual()
        flagged=dict()
        leftover=dict() #Stores the ID of required projects that have not
    ↪  reached the size of the larger of their parents' groups as keys,
    ↪  with that size as values
        projects1=set(self.projects)
        projects2=set(other.projects)
        #Set of projectIDs of projects in both parents
        required=projects1&projects2
        #List of projects in only one parent
        optionals=list(projects1^projects2)
        optionals=list(map(lambda x:(self.projects|other.projects)[x],
    ↪ optionals))

        #Assign a group to every project that appears in both parents
        while len(required)>0:
            #Projects get groups in a random order
            projectID=random.choice(list(required))
            required.remove(projectID)
            project1=self.projects[projectID]
            project2=other.projects[projectID]
            newProject=project1.deepcopy()
```

```
156             #Combine student lists and remove duplicates
157             studentOptions=set(project1.group)|set(project2.group)
158             #Remove students in a different group by only keeping
    ↪ students that remain in the students set
159             studentOptions&=students
160 
161             #Fill the group to the size of the smaller parent group
162             minSize=min(len(project1.group),len(project2.group))
163             maxSize=max(len(project1.group),len(project2.group))
164             if len(studentOptions)>=minSize:
165                 temp=list(studentOptions)
166                 random.shuffle(temp)
167                 group=temp[:minSize]
168                 #Prevent the grouped students from being grouped to
    ↪ another project
169                 students-=set(group)
170                 newProject.addStudent(group)
171                 if maxSize>len(group):
172                     leftover[projectID]=maxSize
173 
174             #In case too many students from the parent groups have
    ↪ already been assigned to different projects
175             else:
176                 students-=studentOptions
177                 newProject.addStudent(list(studentOptions))
178                 flagged[projectID]=minSize-len(studentOptions)
179 
180             individual.addProject(newProject)
181 
182         #Ensure every required project has been filled
183         while len(flagged)>0:
184             projectID=random.choice(list(flagged.keys()))
185             size=flagged[projectID]
186             del flagged[projectID]
187             temp=list(students)
188             random.shuffle(temp)
189             group=temp[:size]
190             students-=set(group)
191             individual.projects[projectID].addStudent(group)
192 
193 
194 
195         while len(students)>0:
196             optionals=get_optionals(optionals,len(students))
197 
198             projectList=list(leftover)+optionals
199 
200             #Incase an individual does not group every student
201             if len(projectList)<1:
202                 individual.fitness=-1
```

```
203                 break
204
205             project=random.choice(projectList)
206
207             #If the project does not yet have a group, add minSize number
    ↪  of students
208             if project in optionals:
209                 newProject=project.deepcopy()
210                 optionals.remove(project)
211                 temp=list(students)
212                 random.shuffle(temp)
213                 group=temp[:project.minSize]
214                 students-=set(group)
215                 newProject.addStudent(group)
216                 individual.addProject(newProject)
217
218                 #If project is not full, allow for it to get more
    ↪ students
219                 if project.minSize!=project.maxSize:
220                     leftover[project.ID]=project.maxSize-project.minSize
221
222             #If project already had a group, add one more student to it
223             else:
224                 projectID=project
225                 student=random.choice(list(students))
226                 individual.projects[projectID].addStudent(student)
227                 students.remove(student)
228
229                 #Remove from dict if now full
230                 leftover[projectID]-=1
231                 if leftover[projectID]<=0:
232                     del leftover[projectID]
233
234
235         return individual
236
237     def mutate(self):
238         return self
239
240     def __str__(self):
241         string=""
242         for i in self.projects:
243             string+= self.projects[i].__str__()+"\n"
244         return string
245
246     #Returns the names, size, gender count, and skill count for each
    ↪ project as a dict with the projects' IDs as keys
247     def get_data(self):
248         data=dict()
249         for project in self.projects:
```

```
            project=self.projects[project]

            genders,skills,students=project.get_data()
            projectData={"name":project.name,"students":students,"size":
    ↪ len(project.group),"genders":genders,"skills":skills}

            data[project.ID]=projectData

        return data



#=====SUBROUTINES=====
#Fetch student details from database
def get_students(database,IDs):
     students=[]

     with sql.connect(database) as conn:
          cur = conn.cursor()

          for ID in IDs:
               #Get each students' skills
               cur.execute("SELECT Skill FROM StudentSkills WHERE
    ↪ StudentID = ?",(ID,))
               skills=set()
               result=cur.fetchall()
               for skill in result:
                    skills.add(skill[0])

               #Get each students' name and gender
               cur.execute("SELECT Name, Gender from Students WHERE
    ↪ StudentID = ?",(ID,))
               entry=cur.fetchall()[0]

               #Create a Student object for each student with their name,
    ↪  skills, and gender
               students.append(Student(ID,entry[0],skills,entry[1]))

     return students


#Fetch project details from database
def get_projects(database,IDs):
     projects={"Optional":[],"notOptional":[]}

     with sql.connect(database) as conn:
          cur = conn.cursor()

          for ID in IDs:
               #Get each projects' skill requirements
```

```
296                 cur.execute("SELECT Skill FROM ProjectSkills WHERE
    ↪ ProjectID = ?",(ID,))
297                 skills=set()
298                 result=cur.fetchall()
299                 for skill in result:
300                       skills.add(skill[0])
301
302                 #Get each projects' data other than skills
303                 cur.execute("SELECT Name, MaxSize, MinSize, Optional from
    ↪ Projects WHERE ProjectID = ?",(ID,))
304                 entry=cur.fetchall()[0]
305
306                 #Create a Project object for each project with its name,
    ↪ skills, maxSize, minSize, and whether or not it is optional
307                 if bool(entry[3]):
308                       projects["Optional"].append(Project(ID,entry[0],
    ↪ skills,entry[1],entry[2],entry[3]))
309                 else:
310                       projects["notOptional"].append(Project(ID,entry[0],
    ↪ skills,entry[1],entry[2],entry[3]))
311
312
313      return projects
314
315 #find optional projects whose minSize is not too large to house the
    ↪ remaining ungrouped students
316 def get_optionals(optionals,extraSlots):
317      optionals.sort(key=lambda x:x.minSize, reverse=True)
318      for i in range(len(optionals)):
319           if optionals[i].minSize>extraSlots:
320                 optionals.pop(i)
321           else:
322                 break
323      return optionals
324
325
326 #Creates an individual for the first population
327 def GenInitialGroups(projects,students):
328      individual=Individual()
329      leftover=[] #Stores projects that have groups, but do not have the
    ↪ maximum number of students assigned yet
330
331      #Fill every required project to minSize with random students
332      required=projects["notOptional"]
333      for i in range(len(required)):
334           required[i]=required[i].deepcopy()
335      for project in required:
336           random.shuffle(students)
337           project.addStudent(students[:project.minSize])
338           individual.addProject(project)
```

```
339
340           #Remove grouped students
341           students=students[project.minSize:]
342
343           #Keep projects with room to spare so that they may take on more
      ↪  students later
344           if project.minSize!=project.maxSize:
345                leftover.append(project.ID)
346
347      #Ensure every student is grouped
348      optionals=[] #Stores projects that do not yet have a group assigned
349      for i in range(len(projects["Optional"])):
350           optionals.append(projects["Optional"][i].deepcopy())
351
352      while len(students)>0:
353           #Only allow students to be grouped to a project with a minSize
      ↪ that allows the remaining students to be housed
354           optionals=get_optionals(optionals,len(students))
355
356           #Incase an individual does not group every student
357           if len(leftover+optionals)<1:
358                individual.fitness=-1
359                break
360           #Add random students to a random project
361           project=random.choice(leftover+optionals)
362           random.shuffle(students)
363
364           #If the project has no group currently, add minSize number of
      ↪ students
365           if type(project)==Project:
366                optionals.remove(project)
367                project.addStudent(students[:project.minSize])
368                individual.addProject(project)
369
370                students=students[project.minSize:]
371
372                #If project is not full, allow for it to gain more
      ↪ students
373                if project.minSize!=project.maxSize:
374                     leftover.append(project.ID)
375
376           #If project already has a group, add one student to it
377           else:
378                individual.projects[project].addStudent(students[0])
379                students.pop(0)
380
381                #Remove from list if now full
382                if len(individual.projects[project].group)>=individual.
      ↪ projects[project].maxSize:
383                     leftover.remove(project)
```

```
384         return individual
385
386
387
388
389
390 #Runs the genetic algorithm
391 def run(database, studentIDs, projectIDs, TARGET_FITNESS, MAX_ITERATIONS,
        ↪  POPULATION_SIZE):
392         #List of students
393         students=get_students(database,studentIDs)
394         #Two lists of projects in dictionary, separated by whether or not
        ↪ the project is optional
395         projects=get_projects(database,projectIDs)
396
397         #Ensure there aren't too many or too few students to meet
        ↪ requirements#
398         numStudents=len(students)
399         minStudents=0
400         maxStudents=0
401
402         #Find max and min amount of students required
403         for project in projects["notOptional"]:
404             maxStudents += project.maxSize
405             minStudents += project.minSize
406         for project in projects["Optional"]:
407             maxStudents += project.maxSize
408         #Return error for too few
409         if numStudents<minStudents:
410             return {"status":-1,"error":f"Error: Only {len(students)} to
        ↪ fill a minimum of {minStudents} places."}
411         #Return error for too many
412         elif numStudents>maxStudents:
413             return {"status":-1,"error":f"Error: {len(students)} is too
        ↪ many to fit in {maxStudents} places."}
414
415         #Generate initial groupings and average fitness
416         population = []
417         for i in range(POPULATION_SIZE):
418             individual = GenInitialGroups(projects, students)
419             individual.calcFitness()
420             population.append(individual)
421
422         #=====Genetic algorithm======
423         for i in range(MAX_ITERATIONS):
424             population.sort(key=lambda x:x.fitness,reverse=True)
425             #Ends early when an individual has reached the target fitness
426             if population[0].fitness>=TARGET_FITNESS:
427                 return {"status":0,"results":population[0].get_data(),"all
        ↪ ":population}
```

```
428
429           #Eliminate individuals in the bottom half in terms of fitness
430           population = population[:POPULATION_SIZE//2]
431           new_pop = []
432
433           #Create next population
434           crossedSize=int(0.9*POPULATION_SIZE)
435           for j in range(crossedSize):
436                #Crossover random individuals
437                parents=random.sample(population,2)
438                individual=parents[0].crossover(parents[1],students)
439
440                #Random mutations to prevent convergence towards a local
    ↪ best solution
441                individual=individual.mutate()
442                individual.calcFitness()
443
444                new_pop.append(individual)
445
446           #Create the next population with the crossover population and
    ↪ the best 10% of the last population
447           population=population[:(POPULATION_SIZE-crossedSize)]
448           population+=new_pop
449      population.sort(key=lambda x:x.fitness,reverse=True)
450      return {"status":0,"results":population[0].get_data(),"all":
    ↪ population}
451
452
453
454 #=====TESTING======
455 if __name__=="__main__":
456      result=run("Code/database.db",[1,3,4,5,6],[4,5,6],0.9,5,50)
457      if result["status"]==0:
458           pop=result["all"]
459           for ind in pop:
460                print(ind.fitness)
461                print(ind)
```

3.3 CreateTables.sql

```
1 -- SQLite
2 PRAGMA foreign_keys = ON;
3
4 CREATE TABLE IF NOT EXISTS Students (
5     StudentID INTEGER PRIMARY KEY,
6     Name TEXT,
7     Gender TEXT
```

```
);

CREATE TABLE IF NOT EXISTS StudentSkills (
    StudentID INTEGER NOT NULL,
    Skill TEXT NOT NULL,
  PRIMARY KEY (StudentID, Skill),
  FOREIGN KEY (StudentID) REFERENCES Students (StudentID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Projects (
    ProjectID INTEGER PRIMARY KEY,
    Name TEXT,
    MaxSize INTEGER,
    MinSize INTEGER,
    Optional INTEGER
);

CREATE TABLE IF NOT EXISTS ProjectSkills (
    ProjectID INTEGER NOT NULL,
    Skill TEXT NOT NULL,
  PRIMARY KEY (ProjectID, Skill),
  FOREIGN KEY (ProjectID) REFERENCES Projects (ProjectID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Results (
    ResultID INTEGER PRIMARY KEY,
    ProjectID INTEGER NOT NULL,
    LocalTimeGenerated TEXT,
    EpochTimeGenerated INTEGER,
    FOREIGN KEY (ProjectID) REFERENCES Projects (ProjectID)
        ON DELETE CASCADE
        ON UPDATE CASCADE

);

CREATE TABLE IF NOT EXISTS ResultStudentLink (
    ResultID INTEGER NOT NULL,
    StudentID INTEGER NOT NULL,
  PRIMARY KEY (ResultID, StudentID),
    FOREIGN KEY (ResultID) REFERENCES Results (ResultID)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
  FOREIGN KEY (StudentID) REFERENCES Students (StudentID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

4 Testing

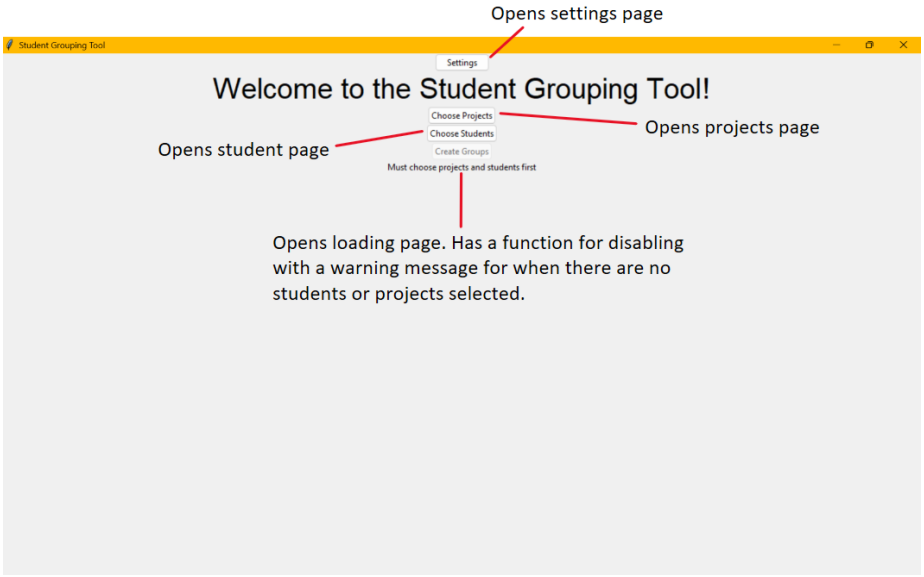


Figure 21: The home page

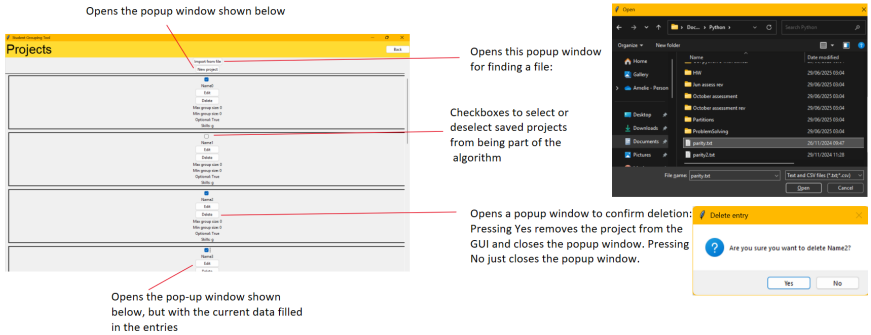


Figure 22: The project page

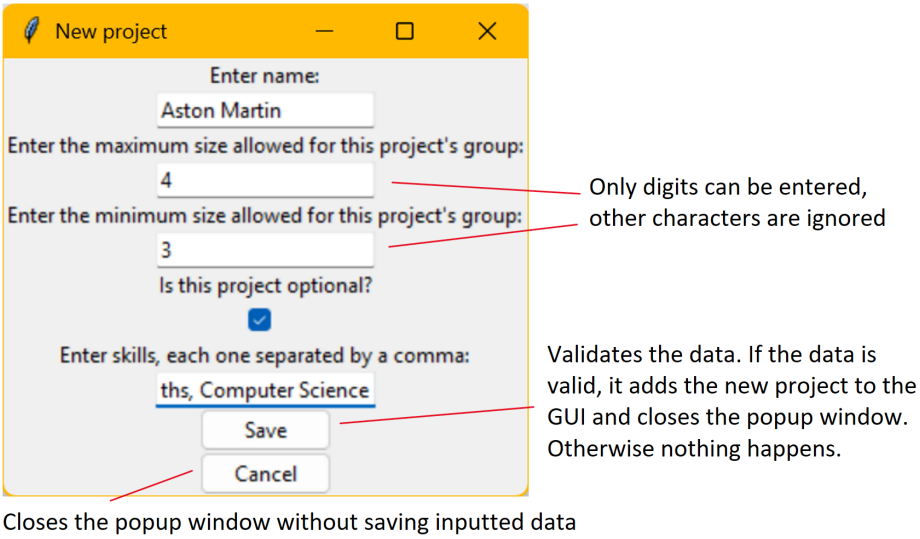


Figure 23: The popup window for entering data about a new project

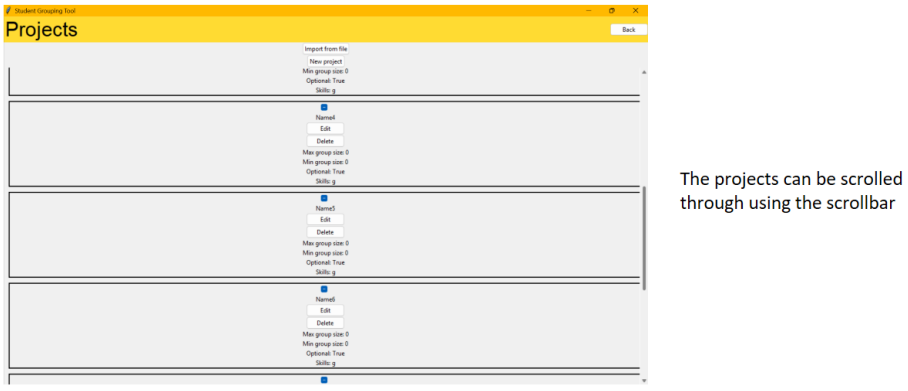


Figure 24: The project page, scrolled down

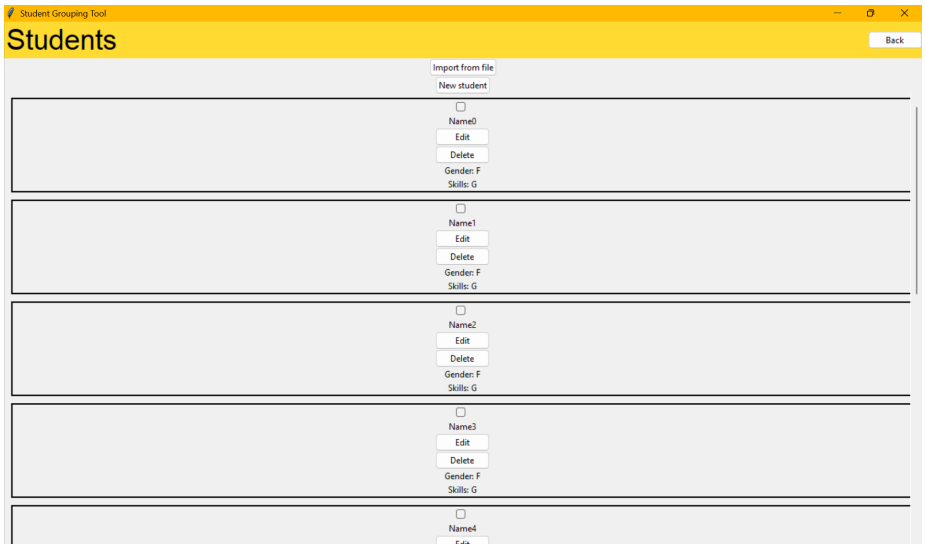


Figure 25: The student page. Functions identically to the projects page

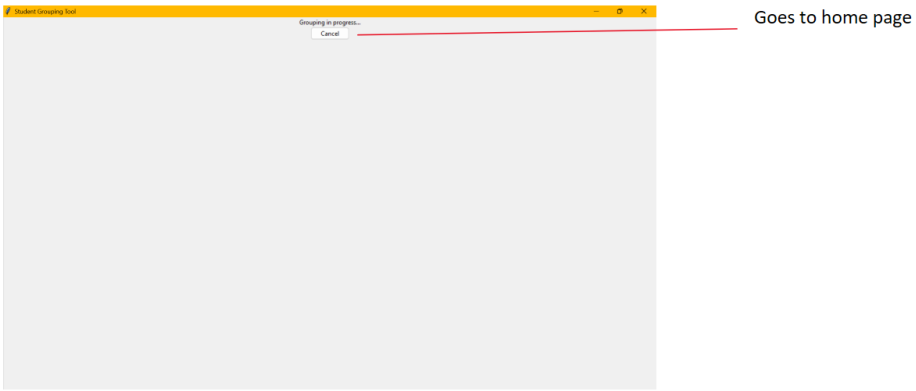


Figure 26: The loading page. It runs the genetic algorithm on page load and opens the results page when results have been returned.

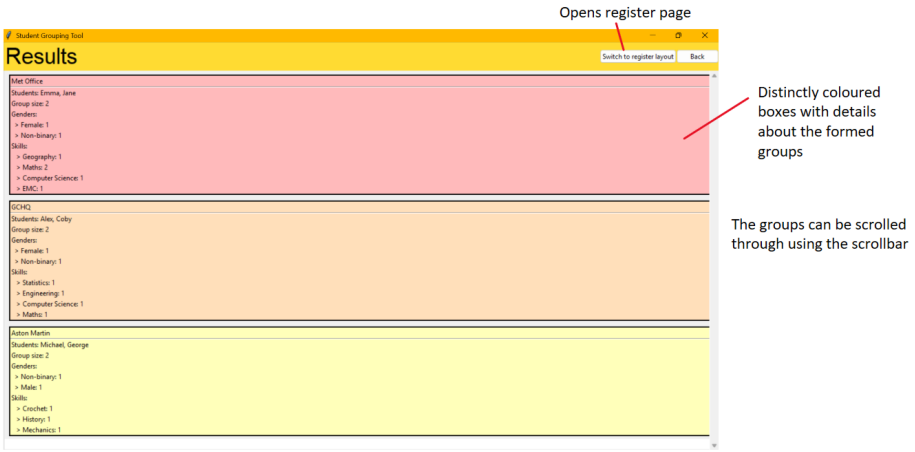


Figure 27: The results page

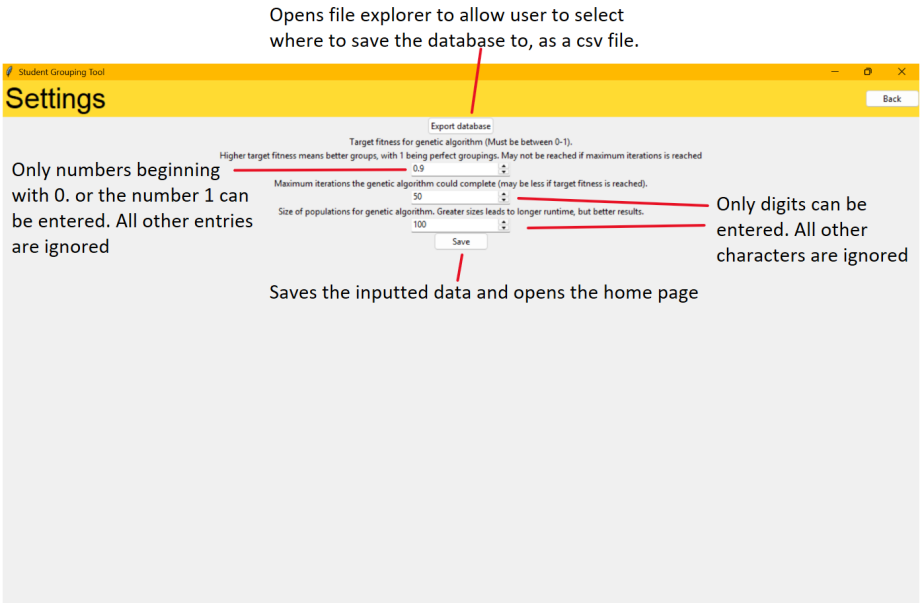


Figure 28: The settings page

4.1 Video

4.2 Testing plan

ID	Obj.	Description	Test Data	Expected Results	Time stamp
1	1a,1e	Check that all student fields are required when entered manually	Name: Tina Gender: Female Skills: Physics, Chemistry 1.1,1.2,1.3 (Erroneous): Name, gender, or skills box is blank 1.4 (Normal): All data above is entered	1.1,1.2,1.3: A warning message displays to the user, the pop-up window remains open, and the data does not get saved. 1.4: The pop-up window closes, the data is displayed on the GUI and saved to the database.	
2	1a,1e,4a	Check that the entered student data is saved when all fields are completed correctly	(Normal): Same as 1.4	An SQL SELECT statement returns the newly added data.	
3	1h, 4a	Check that student data can be edited	(Normal): Edit Tina Name: Dylan Gender: Male Skills: Chemistry, Maths, Biology	An SQL SELECT statement only shows the new data, none of the old data.	
4	1h, 4a	Check that student data can be deleted	(Normal): Delete Dylan	An SQL SELECT statement shows that no data about Dylan exists	
5	1c, 1d	Check that student data is saved from previous uses of the program	(Normal): Close and re-open the program	The data shown on the student page is exactly the same as how it was left	
6	1bi, 1e, 4a	Check that program only accepts CSVs for student data in the required format	6.1 (Erroneous): 6.2 (Normal):	6.1: A warning message displays and the data does not get saved. 6.2: The data is displayed on the GUI and saved to the database.	

ID	Obj.	Description	Test Data	Expected Results	Time stamp
8	1a,1fi	Check that the project's maximum group size must be greater than or equal to minimum group size when entered manually	8.1 (Erroneous): 8.2 (Boundary): 8.3 (Normal):	8.1: A warning message displays to the user, the pop-up window remains open, and the data does not get saved. 8.2: The pop-up window closes, the data is displayed on the GUI and saved to the database. 8.3: The pop-up window closes, the data is displayed on the GUI and saved to the database.	
9	1a,1fii	Check that the project's minimum group size must be greater than 0 when entered manually			
10	1a,1f	Check that all project fields are required			
11	1a,1f,4a	Check that the entered project data is saved when all fields are completed correctly			
12	1h,4a	Check that project data can be edited			
13	1h,4a	Check that project data can be deleted			
14	1c	Check that project data is saved from previous uses of the program			

15	1bi,1d	Check that project data can be imported through csv files when all fields are correct			
16	1bi,1fi	Check that the project's maximum group size must be greater than minimum group size when importing a csv file			
17	1bi,1fii	Check that the project's minimum group size must be greater than 0 when importing a csv file			
18	1bi,1f	Check that all project fields are required when importing a csv file			
19	1gi	Check that setting's target fitness must be between 0 and 1			
20	1gii	Check that setting's max iterations must be a positive integer, not including 0			
21	1giii	Check that setting's size of populations must be a positive integer, not including 0			
22	4bi	Check that database can be exported as a csv file			

23	4c	Check that settings data is stored as a JSON			
24	1i	Check that at least one student, one project, target fitness, max iterations, and size of populations must all be entered before the algorithm is run			

5 Evaluation

5.1 Objective evaluation

The program successfully takes in student and project data, validates it, runs a grouping algorithm, and outputs the results onto a GUI.

[Testing showed that this objective was achieved/not achieved for multiple objectives]

5.2 Third-party feedback

1.

5.3 Feedback evaluation

5.4 Improvements

[Optional objectives] [More input data] [More control over input data] [More input methods] [Exporting outputs] [Design] [Storing results db] Don't do too many, pick 2-4 ig.